

INFORMATIQUE • 1ÈRE E

MODULE 2 : ALGORITHMIQUE ET PROGRAMMATION

LEÇON N10

PROGRAMMATION EN LANGAGE
C

DURÉE CONSEILLÉE : 6 H

DANS CETTE LEÇON

Cette leçon initie les élèves à la programmation procédurale en langage C : installation d'un environnement de développement (IDE), structure d'un programme, utilisation des bibliothèques et des fonctions d'entrée/sortie, structures répétitives et tableaux. Elle concrétise en machine les algorithmes étudiés en algorithmique.

- À la fin de cette leçon, je sais installer et configurer un IDE pour le langage C (Code ::Blocks, Dev-C++)
- À la fin de cette leçon, je sais écrire la structure minimale d'un programme C (directives, fonction main, accolades, point-virgule)
- À la fin de cette leçon, je sais inclure les bibliothèques `stdio.h`, `stdlib.h`, `math.h` et `conio.h` et citer une fonction de chacune
- À la fin de cette leçon, je sais utiliser `printf` et `scanf` avec les formats `%d`, `%f`, `%c` et l'opérateur d'adresse `&`
- À la fin de cette leçon, je sais écrire les structures répétitives `for`, `while` et `do...while` en C
- À la fin de cette leçon, je sais déclarer et manipuler un tableau à une dimension en C

TÉLÉCHARGE
ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Environnement de développement (IDE) et premier programme
2. Bibliothèques et fonctions d'entrée/-sortie
3. Variables, types et mode d'adressage
4. Les structures répétitives : for, while, do...while
5. Les tableaux à une dimension
6. Compilation, tests et correction d'un programme
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais installer et configurer un IDE pour le langage C (Code ::Blocks, Dev-C++)
- À la fin de cette leçon, je sais écrire la structure minimale d'un programme C (directives, fonction main, accolades, point-virgule)
- À la fin de cette leçon, je sais inclure les bibliothèques `stdio.h`, `stdlib.h`, `math.h` et `conio.h` et citer une fonction de chacune
- À la fin de cette leçon, je sais utiliser `printf` et `scanf` avec les formats `%d`, `%f`, `%c` et l'opérateur d'adresse `&`
- À la fin de cette leçon, je sais écrire les structures répétitives `for`, `while` et `do...while` en C
- À la fin de cette leçon, je sais déclarer et manipuler un tableau à une dimension en C
- À la fin de cette leçon, je sais compiler, exécuter et tester un programme dans un IDE en corrigeant les erreurs signalées
- À la fin de cette leçon, je sais distinguer passage de paramètre par valeur et par adresse (mode d'adressage)

PRÉREQUIS

- Notions d'algorithmique : variables, types, affectation, instructions conditionnelles et boucles (leçons précédentes du module 2)

- Traduction d'un algorithme simple en pseudo-code
- Notion de fichier et de dossier sous Windows (environnement informatique)
- Opérations arithmétiques de base et priorités opératoires

1 ENVIRONNEMENT DE DÉVELOPPEMENT (IDE) ET PREMIER PROGRAMME

Situation d'accroche. Le lycée de Nkongsamba veut informatiser le calcul des moyennes de ses élèves de Première. Jusqu'ici, le surveillant général saisit les notes à la main dans un cahier, ce qui provoque des erreurs de report, des oublis et, à chaque fin de trimestre, des contestations de bulletins. Un ancien élève de l'établissement, aujourd'hui étudiant en informatique, propose d'écrire un petit programme en langage C : ce programme lira les notes saisies au clavier, calculera automatiquement la moyenne et affichera le bulletin à l'écran. Mais avant d'arriver là, plusieurs questions se posent : quel logiciel faut-il installer pour écrire un programme C ? Comment transformer le texte du programme en un fichier que l'ordinateur peut exécuter ? Quelle est la forme minimale d'un programme C ?

Problématique. Comment installer et utiliser un environnement de développement pour écrire, compiler et exécuter un premier programme en langage C ?

1.1 Qu'est-ce qu'un IDE ?

Pour écrire une lettre, tu utilises un logiciel de traitement de texte. Pour écrire un programme, un simple éditeur de texte ne suffit pas : l'ordinateur ne comprend que le **langage machine** (des suites de 0 et de 1), alors que le programmeur écrit dans un langage évolué comme le C. Il faut donc, au minimum, trois outils :

- un **éditeur de code** pour taper le texte du programme (le **code source**) ;
- un **compilateur** pour traduire ce code source en langage machine ;
- des outils de test (dont le **débogueur**) pour repérer et corriger les erreurs.

Installer ces trois outils séparément serait fastidieux. C'est pourquoi on utilise un logiciel unique qui les regroupe tous : l'IDE.

DÉFINITION IDE

Un **IDE** (Integrated Development Environment, en français environnement intégré de développement) est un logiciel qui regroupe, dans une même interface, l'éditeur de code, le compilateur et les outils de test (débogueur, console d'exécution) d'un ou plusieurs langages de programmation.

EXEMPLE Quelques IDE utilisables pour le langage C

- Code :: Blocks : gratuit, léger, livré avec le compilateur MinGW ; très répandu dans les lycées et universités ;
- Dev-C++ : gratuit, simple d'emploi, idéal pour débiter sous Windows ;

- Visual Studio Code : éditeur moderne et gratuit de Microsoft, auquel on ajoute une extension C et un compilateur ;
- CodeLite, Eclipse CDT : autres alternatives gratuites.

Dans la suite de la leçon, nous travaillerons avec Code ::Blocks, mais tout ce qui est dit reste valable avec Dev-C++.

PROPRIÉTÉ Rôle du compilateur

Le **compilateur** est un programme qui traduit l'ensemble du code source (fichier `.c`) en langage machine directement exécutable par le processeur (fichier `.exe` sous Windows). Contrairement à un interpréteur qui traduit ligne par ligne à chaque exécution, le compilateur traduit une fois pour toutes : le programme traduit s'exécute ensuite très rapidement. Le compilateur signale les erreurs de syntaxe avec le numéro de ligne concerné.

LESAVAIS-TU ? Le langage C, un monument de l'informatique

Le langage C a été créé en 1972 par Dennis Ritchie dans les laboratoires Bell (États-Unis), pour réécrire le système d'exploitation UNIX. Plus de cinquante ans après, il reste l'un des langages les plus utilisés au monde : les noyaux de Windows, de Linux et d'Android, les logiciels embarqués des compteurs ENEO ou des téléphones, sont en grande partie écrits en C. Apprendre le C, c'est apprendre la langue dans laquelle les ordinateurs modernes sont construits !

1.2 Installation de Code ::Blocks et création d'un projet

Avant d'écrire la moindre ligne de code, il faut installer l'outil. Voici la démarche complète, telle qu'on la réalise sur les postes d'une salle multimédia de lycée.

MÉTHODE Installer Code ::Blocks sous Windows

1. Ouvrir un navigateur et se rendre sur le site officiel www.codeblocks.org, rubrique Downloads.
2. Télécharger la version avec compilateur inclus : le fichier dont le nom contient `mingw-setup.exe` (par exemple `codeblocks-20.03mingw-setup.exe`). C'est essentiel : sans compilateur, l'IDE ne peut rien traduire.
3. Double-cliquer sur le fichier téléchargé et suivre l'assistant : Next, accepter la licence, garder les composants par défaut, choisir le dossier d'installation, puis Install.

4. Au premier lancement, Code ::Blocks détecte automatiquement le compilateur GNU GCC : cliquer sur OK.
5. Vérifier l'installation : menu Settings → Compiler, l'onglet doit indiquer GNU GCC Compiler.

ATTENTION Erreur fréquente à l'installation

Beaucoup d'élèves téléchargent la version sans compilateur (le fichier sans mingw dans son nom). Résultat : à la compilation, le message « can't find compiler executable » apparaît. Solution : réinstaller la version mingw-setup, ou indiquer manuellement le chemin du compilateur dans Settings → Compiler → Toolchain executables.

Une fois l'IDE installé, on ne crée pas un fichier « au hasard » : on crée un **projet console**, c'est-à-dire un programme qui s'exécute dans une fenêtre texte (la **console**), sans interface graphique. C'est le type de programme adapté à notre niveau.

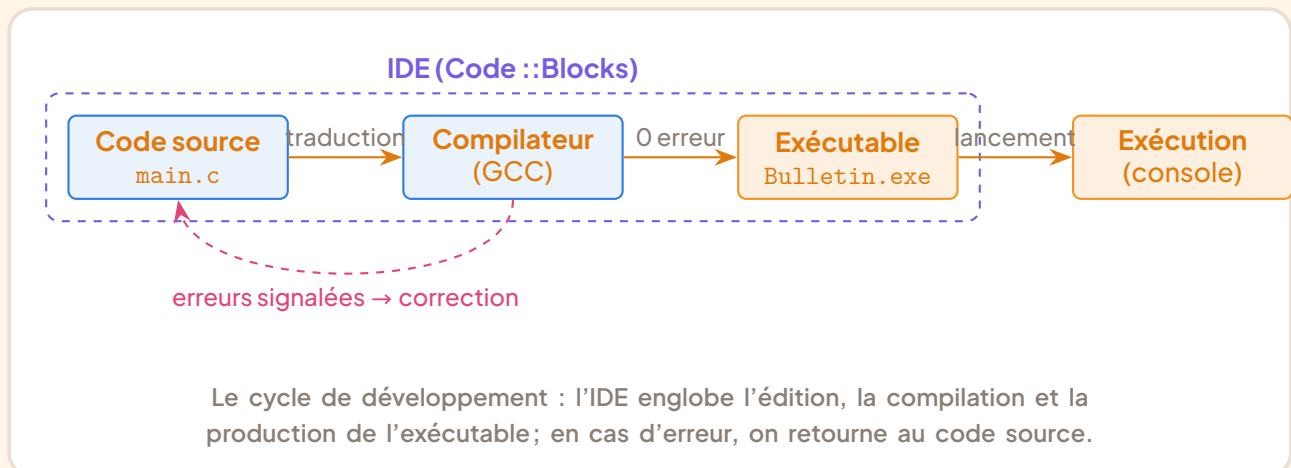
MÉTHODE Créer un nouveau projet console dans Code ::Blocks

1. Menu File → New → Project..., choisir Console application, cliquer Go puis Next.
2. Sélectionner le langage C (et non C++), puis Next.
3. Donner un titre au projet, par exemple Bulletin, choisir le dossier de rangement (par exemple Mes documents\ProgrammesC), puis Next.
4. Laisser le compilateur GNU GCC et les configurations Debug/Release cochées, cliquer Finish.
5. Dans l'arborescence à gauche, ouvrir Sources → main.c : l'éditeur affiche un squelette de programme prêt à compléter.

1.3 Le cycle de développement d'un programme C

Écrire un programme, ce n'est pas taper du texte puis « espérer ». C'est suivre un cycle précis, que tout programmeur répète des dizaines de fois par jour :

1. **Écrire** le code source dans l'éditeur (fichier d'extension .c) ;
2. **Compiler** (menu Build) : le compilateur traduit et signale les erreurs ;
3. **Corriger** les erreurs signalées, puis recompiler jusqu'à obtenir 0 error ;
4. **Exécuter** (menu Run) : le fichier exécutable .exe est lancé et la console affiche les résultats ;
5. **Tester** avec plusieurs jeux de données pour vérifier que les résultats sont corrects.



1.4 Structure minimale d'un programme C

Tout programme C, du plus petit au plus grand, respecte une architecture fixe. Observons-la sur le programme le plus simple possible.

EXEMPLE Premier programme : afficher un message

```
/* Mon premier programme en C
   Affiche un message de bienvenue à l'écran */
#include <stdio.h> // bibliothèque des entrées/sorties

int main()          // fonction principale : le programme commence ici
{
    printf("Bonjour la classe !\n"); // affiche le message
    return 0;        // signale que le programme s'est bien terminé
}
```

Analysons chaque élément, car chacun est indispensable :

- `#include <stdio.h>` : une **directive de préprocesseur** qui demande d'inclure la **bibliothèque standard des entrées/sorties** (standard input/output). Sans elle, la fonction `printf` est inconnue du compilateur. Nous détaillerons les bibliothèques dans la section suivante.
- `int main()` : la **fonction principale**. Tout programme C possède **une et une seule** fonction `main` : c'est par elle que l'exécution commence. Le mot `int` indique qu'elle renvoie un nombre entier au système.
- Les **accolades** `{ }` délimitent le corps de la fonction : toutes les instructions entre `{` et `}` appartiennent à `main`.
- `printf("...")` : l'instruction qui **affiche** le texte à l'écran. Le symbole `\n` provoque un saut de ligne.
- Le **point-virgule** ; termine **chaque instruction**. C'est la ponctuation du langage C : comme le point en fin de phrase en français.

- `return 0 ;` : la fonction `main` renvoie la valeur 0 au système d'exploitation, ce qui signifie par convention « tout s'est bien passé ».

PROPRIÉTÉ Commentaires et casse en langage C

- Un **commentaire** est un texte ignoré par le compilateur, destiné au lecteur humain. Il en existe deux formes : `//` pour un commentaire jusqu'à la fin de la ligne, et `/* ... */` pour un commentaire sur plusieurs lignes.
- Le langage C est **sensible à la casse** : il distingue les majuscules des minuscules. Ainsi `main`, `Main` et `MAIN` sont trois noms différents. Écrire `Printf` au lieu de `printf` provoque une erreur de compilation.

ATTENTION Les deux erreurs classiques du débutant

- Oublier le point-virgule en fin d'instruction : le compilateur signale alors une erreur de syntaxe, souvent à la ligne suivante, car c'est là qu'il découvre que l'instruction précédente n'était pas terminée. Cherche donc l'erreur une ligne au-dessus de celle indiquée !
- Oublier une accolade fermante `}` : le compilateur signale une erreur en fin de fichier (« expected '}' at end of input »). Compte toujours tes accolades : autant d'ouvrantes que de fermantes.



Capture stylisée d'un IDE : en haut l'éditeur de code, en bas à gauche les messages du compilateur, en bas à droite la console de sortie.

1.5 Travaux pratiques : écrire, compiler et exécuter « Bonjour le Cameroun ! »

EXPÉRIENCE TP guidé : mon premier programme C

Objectif : installer le réflexe complet du cycle de développement en créant un programme qui affiche `Bonjour le Cameroun !`.

Matériel et logiciels : un poste sous Windows, Code ::Blocks installé avec le compilateur MinGW.

Manipulations :

1. Lancer Code ::Blocks et créer un projet console en C nommé `Salutation` (méthode vue plus haut).
2. Ouvrir `main.c` et compléter le code comme suit :

```
#include <stdio.h> // bibliotheque d'entrees/sorties

int main()
{
    // Affichage du message de bienvenue
    printf("Bonjour le Cameroun !\n");
    printf("Bienvenue au lycee de Nkongsamba.\n");
    return 0;
}
```

3. Enregistrer (Ctrl+S), puis compiler : menu `Build` → `Build` (ou touche Ctrl+F9 / F8 se-

lon la version). Observer la zone des messages.

4. Exécuter : menu `Build` → `Run`. La console s'ouvre et affiche les deux lignes.
5. Provoquer volontairement une erreur : supprimer le point-virgule de la première ligne `printf`, recompiler, lire le message d'erreur et noter le numéro de ligne indiqué. Corriger, recompiler.
6. Remplacer `printf` par `Printf` (majuscule), recompiler et observer : le compilateur signale une fonction inconnue, car le C distingue les majuscules des minuscules. Corriger.

Observations attendues : la console affiche :

```
Bonjour le Cameroun !  
Bienvenue au lycee de Nkongsamba.
```

L'erreur de point-virgule est signalée à la ligne suivant l'oubli ; l'erreur de casse produit le message « 'Printf' undeclared ».

Interprétation : le compilateur est un correcteur infatigable mais précis : il localise chaque faute. Le cycle écrire → compiler → corriger → exécuter devient alors un réflexe naturel.

EXERCICE RÉSOLU 1 Identifier et corriger les erreurs d'un programme

Énoncé. Un élève de Première D a tapé le programme suivant, mais la compilation échoue. Trouve les trois erreurs et réécris le programme corrigé.

```
#include <stdio.h>  
  
int Main()  
{  
    printf("Moyenne generale : 14,5 / 20\n")  
    return 0;
```

SOLUTION

Solution détaillée.

1. `int Main()` : le C est sensible à la casse ; la fonction principale doit s'écrire `main` en minuscules. Avec `Main`, le compilateur ne trouve aucun point d'entrée du programme.
2. Il manque le point-virgule à la fin de l'instruction `printf(...)`.
3. Il manque l'accolade fermante `}` qui clôt le corps de la fonction `main`.

Programme corrigé :

```
#include <stdio.h>  
  
int main()  
{
```

```
printf("Moyenne generale : 14,5 / 20\n");  
return 0;  
}
```

Ce programme compile sans erreur et affiche : Moyenne generale : 14,5 / 20.

À RETENIR L'essentiel de la section

- Un **IDE** regroupe l'éditeur de code, le compilateur et les outils de test ; Code ::Blocks et Dev-C++ sont des IDE gratuits adaptés au C.
- Le **compilateur** traduit le code source (.c) en exécutable (.exe) et signale les erreurs de syntaxe.
- Le cycle de développement : écrire → compiler → corriger → exécuter → tester.
- Tout programme C contient les directives `#include`, une unique fonction `int main()`, un corps délimité par des accolades, des instructions terminées par `;` et un `return 0` ; final.
- Le C est sensible à la casse ; les commentaires `//` et `/* */` documentent le code sans affecter l'exécution.

EXERCICE 1 À toi de jouer

1. Cite les trois outils regroupés dans un IDE et explique le rôle de chacun en une phrase.
2. Écris un programme C complet qui affiche ton nom, ta classe et ton établissement sur trois lignes distinctes.
3. Que se passe-t-il si l'on écrit `INT MAIN()` à la place de `int main()` ? Justifie.
4. Dans Code ::Blocks, décris la différence entre les commandes `Build` et `Run`.

CORRIGÉ Exercice : À toi de jouer

1. L'éditeur de code permet de taper le code source ; le compilateur traduit le code source en langage machine et signale les erreurs ; le débogueur / les outils de test permettent d'exécuter le programme pas à pas pour repérer les erreurs de logique.
2. Programme possible :

```
#include <stdio.h>
```

```
int main()
{
    printf("Nom : Fotso Amina\n");
    printf("Classe : Premiere C\n");
    printf("Etablissement : Lycee de Nkongsamba\n");
    return 0;
}
```

3. Le compilateur signale une erreur : le C est sensible à la casse, donc `INT MAIN` n'est pas reconnu comme la fonction principale `int main`; le programme n'a plus de point d'entrée.
4. Build lance la compilation : le code source est traduit en exécutable et les erreurs sont signalées, mais le programme n'est pas lancé. Run lance l'exécution du dernier exécutable produit : la console s'ouvre et le programme tourne.

2 BIBLIOTHÈQUES ET FONCTIONS D'ENTRÉE/SORTIE

Dans la section précédente, tu as installé ton IDE et écrit ton premier programme C : la fameuse structure minimale avec `main()`. Tu as peut-être remarqué, sans trop t'y attarder, cette ligne mystérieuse tout en haut du programme : `#include <stdio.h>`. Le moment est venu de percer ce mystère. Car sans elle, impossible d'afficher quoi que ce soit à l'écran ou de lire ce que l'utilisateur tape au clavier. Cette section va te révéler comment un programme C dialogue avec l'extérieur : grâce aux **bibliothèques** et à leurs fonctions d'**entrée/sortie**.

2.1 Qu'est-ce qu'une bibliothèque ?

Situation de départ. Imagine que tu gères la cantine du lycée et que tu veuilles un programme qui calcule la racine carrée du nombre de plats pour organiser les tables carrées. Faut-il que tu écrives toi-même l'algorithme de calcul d'une racine carrée ? Heureusement non ! Des programmeurs ont déjà écrit, testé et optimisé ces fonctions il y a des décennies. Elles sont rangées dans des fichiers que l'on appelle bibliothèques.

DÉFINITION Bibliothèque

Une **bibliothèque** est un fichier d'en-tête, d'extension `.h` (pour header), qui contient des fonctions prêtes à l'emploi : afficher du texte, lire le clavier, calculer une racine carrée, etc. On l'inclut en tête de programme avec la directive :

```
#include <nom_de_la_bibliotheque.h>
```

PROPRIÉTÉ Placement et rôle de `#include`

La directive `#include` est une instruction du préprocesseur : avant même la compilation, elle recopie le contenu du fichier d'en-tête dans ton programme. Elle doit donc être placée tout en haut du fichier, avant `main()`, une par ligne. Sans l'inclusion d'une bibliothèque, l'utilisation de ses fonctions provoque une erreur de compilation (« fonction non déclarée »).

Bibliothèque	Rôle	Fonctions principales
<code>stdio.h</code>	Entrées/sorties standard (écran, clavier)	<code>printf</code> , <code>scanf</code>
<code>stdlib.h</code>	Fonctions utilitaires générales	<code>system</code> , <code>rand</code> , <code>exit</code> , <code>abs</code>
<code>math.h</code>	Calculs mathématiques	<code>sqrt</code> , <code>pow</code> , <code>fabs</code>
<code>conio.h</code>	Entrées/sorties console (Windows)	<code>getch</code>

Les quatre bibliothèques C au programme de Première

2.2 La bibliothèque `stdio.h` : afficher avec `printf`

`stdio.h` (standard input/output) est la bibliothèque indispensable : c'est elle qui permet au programme de « parler » (afficher à l'écran) et d'« écouter » (lire le clavier).

Intuition. La fonction `printf` (print formatted, « afficher avec format ») affiche du texte à l'écran. Mais souvent, on veut afficher la valeur d'une variable au milieu d'une phrase. On place alors dans le texte un **trou** symbolisé par le caractère `%`, et on donne ensuite la variable qui viendra remplir ce trou.

DÉFINITION Les caractères `%` et `&`

Le caractère `%` introduit un format d'affichage ou de saisie : `%d` pour un entier, `%f` pour un réel, `%c` pour un caractère, `%s` pour une chaîne de caractères. Le caractère `&` placé devant le nom d'une variable désigne son adresse mémoire (l'endroit exact où elle est stockée).

PROPRIÉTÉ Syntaxe de `printf`

```
printf("texte avec formats", variable1, variable2);
```

- Chaque format `%...` est remplacé, dans l'ordre, par la valeur de la variable correspon-

dante.

- La séquence `\n` provoque un saut de ligne.
- `%.2f` affiche un réel arrondi à 2 chiffres après la virgule (pratique pour une moyenne!).

EXEMPLE Premiers affichages

```
int age = 16;  
float moyenne = 13.456;  
printf("J'ai %d ans.\n", age);      /* J'ai 16 ans. */  
printf("Moyenne : %.2f\n", moyenne); /* Moyenne : 13.46 */  
printf("Note : %c\n", 'A');        /* Note : A */
```

ATTENTION Formats et types doivent correspondre

Utiliser `%d` pour afficher un `float` (ou `%f` pour un `int`) ne provoque pas toujours d'erreur de compilation, mais affiche des valeurs aberrantes. Retiens le trio : `int` → `%d`, `float` → `%f`, `char` → `%c`.

2.3 Lire le clavier avec `scanf` : le rôle du caractère `&`

Intuition. `scanf` fait l'inverse de `printf` : elle lit ce que l'utilisateur tape au clavier et le range dans une variable. Mais pour ranger une valeur dans une maison, il faut connaître son adresse. C'est exactement le rôle du caractère `&` : `&age` signifie « l'adresse de la case mémoire nommée `age` ». On dit que `scanf` utilise un **mode d'adressage** par adresse : on ne donne pas la valeur de la variable, mais son emplacement, pour que la fonction puisse y écrire directement.

PROPRIÉTÉ Syntaxe de `scanf`

```
scanf("format", &variable);
```

Le format doit correspondre au type de la variable : `%d` pour un `int`, `%f` pour un `float`, `%c` pour un `char`.

```
scanf("%d", &age);
```

écrit la valeur tapée
age moyenne ...



`&age` désigne l'adresse de la case mémoire : `scanf` y dépose la valeur saisie

MÉTHODE Lire correctement une variable avec `scanf`

1. Identifier le type de la variable à remplir (`int`, `float`, `char`).
2. Choisir le format correspondant : `%d`, `%f` ou `%c`.
3. Écrire `scanf("format", &variable);` en n'oubliant jamais le `&` devant le nom de la variable.
4. Afficher avant la saisie un message d'invite avec `printf` pour guider l'utilisateur.

ATTENTION Erreur fréquente n°1 : oublier le `&`

Écrire `scanf("%d", age);` sans le `&` est l'erreur classique du débutant : `scanf` reçoit la valeur de `age` au lieu de son adresse, et tente d'écrire à un emplacement mémoire invalide. Résultat : le programme plante (fenêtre qui se ferme brutalement, erreur de segmentation). De même, saisir un `float` avec `%d` fausse complètement la valeur lue.

2.4 Les autres bibliothèques : `stdlib.h`, `math.h`, `conio.h`

stdlib.h (standard library) fournit des fonctions utilitaires : `system("cls")` efface l'écran de la console (sous Windows), `rand()` génère un nombre pseudo-aléatoire, `exit(0)` termine immédiatement le programme, `abs(x)` renvoie la valeur absolue d'un entier.

math.h contient les fonctions mathématiques : `sqrt(x)` (racine carrée), `pow(x, y)` (x^y), `fabs(x)` (valeur absolue d'un réel). Indispensable pour les calculs scientifiques des séries C, D et E. Note : pour utiliser ces fonctions, il faut parfois lier explicitement la bibliothèque mathématique à la compilation (option `-lm` sous GCC), mais les IDE scolaires (Code::Blocks, Dev-C++) le font automatiquement.

conio.h (console input/output, disponible sous Windows) fournit `getch()`, qui lit un caractère au clavier sans l'afficher et sans attendre la touche Entrée. Placé juste avant `return 0;`, il force le programme à attendre une touche : la fenêtre console reste ouverte et tu peux lire tes résultats.

tats.

LESAVAIS-TU ? Pourquoi la fenêtre se ferme-t-elle trop vite ?

Quand tu lances ton programme depuis l'IDE, la console s'ouvre, exécute le programme en une fraction de seconde, puis se ferme aussitôt : impossible de lire le résultat ! La fonction `getch()` de `conio.h` résout ce problème : elle met le programme en pause jusqu'à ce qu'une touche soit pressée. C'est l'astuce utilisée par tous les élèves camerounais qui programment en C sous Windows avec Code ::Blocks ou Dev-C++.

EXERCICE RÉSOLU 2 Moyenne de deux notes

Énoncé. Écrire un programme C complet qui demande à un élève ses deux notes de séquence (réels), calcule sa moyenne et l'affiche avec deux chiffres après la virgule.

SOLUTION

Solution détaillée.

```
#include <stdio.h> /* printf et scanf */
#include <stdlib.h> /* fonctions utilitaires */
#include <conio.h> /* getch : pause ecran */

int main()
{
    float note1, note2, moy; /* declaration des variables */

    printf("Entrez la premiere note : ");
    scanf("%f", &note1); /* %f car float, & obligatoire */

    printf("Entrez la deuxieme note : ");
    scanf("%f", &note2);

    moy = (note1 + note2) / 2; /* calcul de la moyenne */

    printf("Votre moyenne est : %.2f\n", moy);

    getch(); /* pause : la fenetre reste ouverte */
    return 0;
}
```

Trace d'exécution : si l'utilisateur saisit 12 puis 15, le programme calcule $moy = (12 + 15) / 2 = 13,5$ et affiche : Votre moyenne est : 13.50. Remarque l'usage de `%.2f` qui arrondit proprement l'affichage, et le `&` devant chaque variable dans `scanf`.

EXPÉRIENCE TP guidé : saisir et réafficher l'âge et la moyenne d'un élève

Objectif : maîtriser `printf`, `scanf` et le `&`. Matériel : un poste avec Code ::Blocks (ou Dev-

C++) installé.

1. Crée un nouveau projet C nommé `profil_eleve`.
2. Saisis le programme suivant :

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>

int main()
{
    int age;
    float moyenne;

    printf("Quel est votre age ? ");
    scanf("%d", &age);      /* entier : %d */

    printf("Quelle est votre moyenne ? ");
    scanf("%f", &moyenne); /* reel : %f */

    printf("\n--- Fiche de l'eleve ---\n");
    printf("Age : %d ans\n", age);
    printf("Moyenne : %.2f / 20\n", moyenne);

    getch();
    return 0;
}
```

3. Compile (touche F9 sous Code ::Blocks) puis exécute.
4. Observations : saisis 17 puis 14,25 (ou 14.25 selon la configuration régionale). Vérifie que les valeurs sont réaffichées correctement.
5. Test du piège : retire volontairement le `&` devant `age`, recompile et exécute. Observe le plantage, puis corrige.
6. Interprétation : le `&` fournit à `scanf` l'adresse mémoire où ranger la saisie ; sans lui, la fonction ne sait pas où écrire.

À RETENIR L'essentiel de la section

- Une **bibliothèque** (.h) regroupe des fonctions prêtes à l'emploi ; on l'inclut en tête de programme avec `#include <nom.h>`.
- `stdio.h` : `printf` (afficher) et `scanf` (saisir) ; `stdlib.h` : utilitaires (`abs`, `rand...`) ; `math.h` : calculs (`sqrt`, `pow`, `fabs`) ; `conio.h` : `getch()`.
- Le caractère **%** introduit un format : `%d` (entier), `%f` (réel), `%c` (caractère), `%s` (chaîne) ; `\n` saute une ligne.
- Dans `scanf`, le caractère **&** donne l'adresse de la variable (mode d'adressage) : l'ou-

blier fait planter le programme.

EXERCICE 2 À toi de jouer

Écris un programme C qui demande le prix unitaire d'un cahier (réel) et la quantité achetée (entier) à la librairie du lycée, puis affiche le montant total à payer avec deux chiffres après la virgule. Pense aux bibliothèques nécessaires et aux bons formats.

CORRIGÉ Exercice 1

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>

int main()
{
    float prix, total;
    int quantite;

    printf("Prix unitaire du cahier (FCFA) : ");
    scanf("%f", &prix);
    printf("Quantite achetee : ");
    scanf("%d", &quantite);

    total = prix * quantite;
    printf("Montant total : %.2f FCFA\n", total);

    getch();
    return 0;
}
```

Points de vigilance : %f avec &prix (réel), %d avec &quantite (entier), et %.2f pour un affichage monétaire propre.

3

VARIABLES, TYPES ET MODE D'ADRESSAGE

Dans la section précédente, nous avons découvert les fonctions `printf` et `scanf` pour dialoguer avec l'utilisateur. Nous avons vu que `scanf` exigeait un symbole mystérieux `&` devant le nom de la variable. Pourquoi ce symbole ? Que devient l'information une fois stockée ? Comment une fonction peut-elle modifier une variable qui lui est confiée ? C'est tout l'objet de cette section : comprendre comment les données sont organisées en mémoire et comment les fonctions y accèdent.

3.1 Types de base, déclaration et initialisation

En C, toute variable doit être **déclarée** avant d'être utilisée. La déclaration réserve une zone en mémoire (la RAM) et associe un **nom** (l'identificateur) à cette zone. Le **type** détermine la taille de la zone réservée et la manière dont les bits sont interprétés (entier, réel, caractère).

DÉFINITION Types de base en C

Les quatre types fondamentaux (dits scalaire) sont :

- **int** : entier signé (généralement 4 octets sur architecture 32/64 bits modernes). Intervalle typique : [-2 147 483 648; 2 147 483 647].
- **float** : réel simple précision (4 octets, environ 6–7 chiffres significatifs).
- **double** : réel double précision (8 octets, environ 15 chiffres significatifs). C'est le type par défaut pour les constantes réelles (3.14 est un double).
- **char** : caractère unique (1 octet). Stocke un code ASCII/UTF-8 (ex. 'A', '7', '+').

PROPRIÉTÉ Syntaxe de déclaration et d'initialisation

- Déclaration simple : `type identificateur ;` (la valeur est indéterminée, « poubelle »).
- Initialisation à la déclaration : `type identificateur = valeur ;`
- Déclaration multiple : `type id1, id2 = val2, id3 ;`

Les identificateurs respectent les règles : lettres, chiffres, underscore `_`, ne commencent pas par un chiffre, sensible à la casse (`note` ≠ `Note`).

EXEMPLE Déclarations variées

```
#include <stdio.h>

int main() {
    // Entiers
    int age = 17;           // Initialisation
    int nombreEleves;       // Non initialisé (valeur aléatoire)
    nombreEleves = 45;      // Affectation ultérieure

    // Réels
    float moyenne = 12.5f; // Le suffixe 'f' force le type float
    double pi = 3.141592653589793; // Précision supérieure

    // Caractères
    char initiale = 'K';    // Guillemets simples obligatoires
    char toucheEntree = '\n'; // Caractère d'échappement (retour à la ligne)
```

```
// Affichage avec les bons spécificateurs de format
printf("Age : %d ans\n", age);
printf("Moyenne : %.2f / 20\n", moyenne);
printf("PI approx : %.10f\n", pi);
printf("Initiale : %c (code ASCII %d)\n", initiale, initiale);

return 0;
}
```

Remarque : %d pour int, %f pour float/double (en printf, float est promu en double), %c pour char. Le .2f limite l'affichage à 2 décimales.

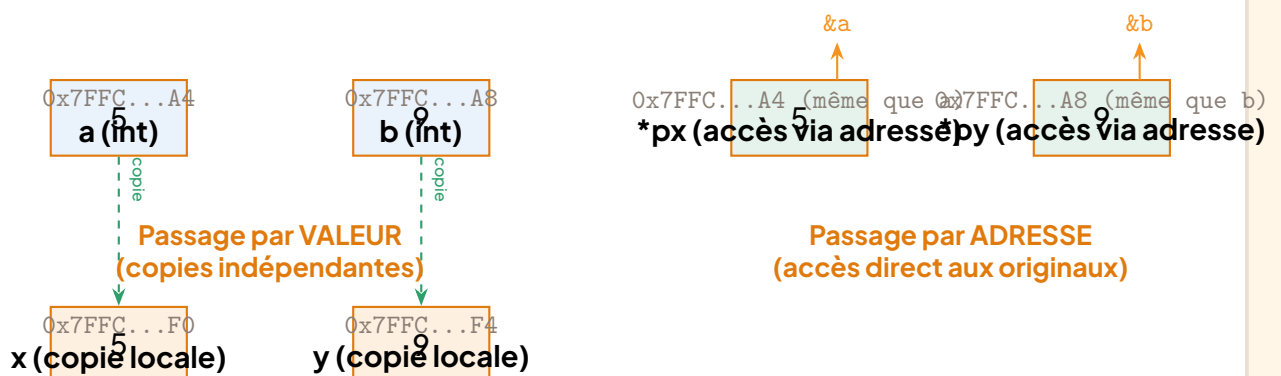
3.2 La mémoire et l'opérateur & (adresse de)

Chaque variable déclarée occupe une ou plusieurs **cases mémoire** contiguës. Chaque case a une **adresse** unique (un numéro d'ordre hexadécimal, ex. 0x7ffd4a3c). L'opérateur & (lu « adresse de ») permet de récupérer cette adresse.

DÉFINITION Opérateur & (adresse de)

Si x est une variable, $\&x$ est une constante (on ne peut pas la modifier) de type « adresse de x » (techniquement un pointeur, notion approfondie en Terminale). Elle indique où se trouve x en mémoire.

C'est pourquoi `scanf` exige `&variable` : la fonction a besoin de savoir où écrire la valeur lue au clavier. Si on oublie le `&`, `scanf` écrit à une adresse aléatoire (contenu de la variable non initialisée) → plantage ou corruption mémoire.



Représentation schématique de la mémoire : deux variables `a=5` et `b=9`. En haut : passage par adresse (les paramètres reçoivent les adresses `&a`, `&b` et accèdent aux cases d'origine). En bas : passage par valeur (des copies locales `x`, `y` sont créées à d'autres adresses).

3.3 Paramètres de fonction et modes d'adressage

Une **fonction** reçoit des données via ses **paramètres formels** (déclarés dans l'en-tête) qui prennent les valeurs des **arguments réels** (fourni à l'appel). Le **mode d'adressage** (ou mode de passage) définit comment cette transmission s'effectue.

DÉFINITION Mode d'adressage (Passage de paramètres)

Le mode d'adressage précise la mécanique de transmission des données entre l'appelant et la fonction appelée :

1. Passage par valeur (par défaut en C) : La fonction reçoit une copie de la valeur de l'argument. Toute modification du paramètre à l'intérieur de la fonction n'affecte que la copie locale ; la variable de l'appelant reste inchangée.
2. Passage par adresse (simulé en C) : L'appelant transmet l'adresse de la variable (&var). Le paramètre formel est alors une variable capable de stocker une adresse (un pointeur). La fonction peut modifier le contenu de la case mémoire d'origine en déréférençant l'adresse (opérateur *).

PROPRIÉTÉ Règle fondamentale en C (admise)

En langage C, tout passage de paramètre se fait par valeur. Pour obtenir l'effet d'un passage par adresse (modifier la variable appelante), on passe explicitement l'adresse de la variable (&variable) et le paramètre formel est déclaré comme un pointeur (ex. `int *p`).

EXEMPLE Échec du passage par valeur : tentative d'échange

```
#include <stdio.h>

// Version INCORRECTE : passage par valeur
void echange_valeur(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
    printf("Dans echange_valeur : x=%d, y=%d\n", x, y); // Affiche 9, 5
}

int main() {
    int a = 5, b = 9;
    printf("Avant appel : a=%d, b=%d\n", a, b); // Affiche 5, 9
    echange_valeur(a, b); // Copies de 5 et 9 transmises
    printf("Après appel : a=%d, b=%d\n", a, b); // Affiche 5, 9 (INCHANGÉ !)
    return 0;
}
```

Ici, `x` et `y` sont des variables locales à `echange_valeur`. Elles meurent à la fin de la fonction.

a et b n'ont jamais été touchées.

EXERCICE RÉSOLU 3 Échange réussi par passage d'adresses

Énoncé : Écrire une fonction `échange` qui inverse réellement le contenu de deux variables entières `a` et `b` du `main`. Tracer l'exécution pas à pas.

Solution détaillée :

```
#include <stdio.h>

// Paramètres formels : pointeurs (adresses)
void échange(int *px, int *py) {
    int temp = *px; // 1. Lit la valeur À L'ADRESSE px (déréférencement *)
    *px = *py;      // 2. Écrit la valeur de py DANS l'adresse px
    *py = temp;     // 3. Écrit temp DANS l'adresse py
}

int main() {
    int a = 5, b = 9;
    printf("1. Avant : a=%d (adresse %p), b=%d (adresse %p)\n", a, &a, b, &b);

    échange(&a, &b); // Appel : on transmet LES ADRESSES

    printf("4. Après : a=%d, b=%d\n", a, b);
    return 0;
}
```

Trace d'exécution détaillée (exécution pas à pas) :

Étape	Action	État mémoire (a, b)	Commentaire
1	int a=5, b=9;	a=5 @0x100, b=9 @0x104	Allocation dans main
2	échange(&a, &b);		px pointe vers a, py vers b
3	temp = *px;	temp = 5	* lit le contenu de 0x100
4	*px = *py;	a=9, b=9	* écrit 9 à 0x100 (a change!)
5	*py = temp;	a=9, b=5	* écrit 5 à 0x104 (b change!)
6	Fin fonc- tion	a=9, b=5	Variables locales px, py, temp détruites

Résultat final : Après : a=9, b=5. L'échange a bien eu lieu car la fonction a écrit directe-

ment dans les cases mémoire de `a` et `b` grâce à leurs adresses.

ATTENTION Piège classique : confusion valeur / adresse

- Erreur : `void f(int x) { x = 10; }` appelé par `f(a)`; → `a` ne change pas.
- Correct : `void f(int *x) { *x = 10; }` appelé par `f(&a)`; → `a` devient 10.
- Erreur `scanf` : `scanf("%d", age);` au lieu de `scanf("%d", &age);`. Le programme lit l'entrée mais l'écrit n'importe où (souvent `Segmentation fault`).
- Oubli du `*` dans la fonction : Recevoir `int *p` mais écrire `p = 10;` (change l'adresse locale, pas la valeur pointée). Il faut `*p = 10;`.

LESAVAIS-TU ? Le `&` et le `*` : un duo inséparable

L'opérateur `&` (adresse de) est l'opérateur de référencement : il fabrique une référence (une adresse) vers une variable. Son inverse est l'opérateur `*` (étoile), appelé opérateur de déréférencement (ou indirection) : `*p` signifie « la valeur stockée à l'adresse contenue dans `p` ».

- `p = &a;` (← `p` reçoit l'adresse de `a`)
- `*p = 20;` (← on met 20 dans la case pointée par `p`, donc dans `a`)

Cette mécanique `&` / `*` est le cœur de la manipulation mémoire en C. Les pointeurs (`int *`, `float *`, etc.) seront le sujet principal de votre programme de Terminale. Maîtriser `&` et le passage d'adresse dès la Première, c'est préparer cet avenir sereinement !

MÉTHODE Comment choisir le mode de passage pour une fonction ?

1. La fonction a-t-elle besoin de modifier la variable de l'appelant ?
 - NON (lecture seule : calcul, affichage, test) → Passage par valeur. Simple, sûr, efficace pour les petits types (`int`, `float`, `char`).
 - OUI (échange, saisie, modification d'état, retour de plusieurs résultats) → Passage par adresse.
2. Si passage par adresse :
 - Dans l'en-tête : type pointeur (`void f(int *p);`).
 - Dans le corps : déréférencer pour lire/écrire (`*p = ...;` ou `x = *p;`).
 - À l'appel : passer l'adresse (`f(&variable);`).

3. Cas particulier : les tableaux. Un nom de tableau est déjà une adresse (adresse du premier élément). On ne met jamais de & devant un nom de tableau pour le passer à une fonction (voir section 5 sur les tableaux).

EXPÉRIENCE**TP guidé : Observer les adresses et tester les passages**

Objectif : Visualiser les adresses mémoire et constater la différence entre passage par valeur et passage par adresse. Matériel : IDE C (Code :: Blocks, Dev-C++, VS Code + GCC) ou compilateur en ligne (OnlineGDB, Replit).

Manipulation :

1. Créez un nouveau projet `TestAdresses.c`.
2. Copiez le code ci-dessous.
3. Compilez et exécutez. Notez les adresses affichées (elles changent à chaque exécution : ASLR).
4. Commentez l'appel `modifier_valeur` et décommentez `modifier_adresse`. Réexécutez.

```
#include <stdio.h>

void afficher_adresses(int v, int *pv) {
    printf(" [Fonction] Valeur recue (copie) : %d @ %p\n", v, (void*)&v);
    printf(" [Fonction] Adresse recue (pointeur) : %p -> valeur %d\n", (void*)pv, *pv);
}

void modifier_valeur(int x) { x = 999; } // Passage par valeur
void modifier_adresse(int *px) { *px = 999; } // Passage par adresse

int main() {
    int variable = 42;
    printf(" --- Dans main ---\n");
    printf("Variable 'variable' = %d @ %p\n", variable, (void*)&variable);

    printf("1. Appel passage par VALEUR :\n");
    afficher_adresses(variable, &variable);
    modifier_valeur(variable);
    printf("Après modifier_valeur : variable = %d (inchangé ?)\n", variable);

    printf("2. Appel passage par ADRESSE :\n");
    afficher_adresses(variable, &variable);
    modifier_adresse(&variable);
    printf("Après modifier_adresse : variable = %d (modifié !)\n", variable);

    return 0;
}
```

Observations attendues :

- L'adresse de `variable` dans `main` (`&variable`) est identique à l'adresse affichée par

`afficher_adresses` pour le pointeur `pv`.

- L'adresse du paramètre `v` (copie) dans `afficher_adresses` est différente de celle de `variable`.
- `modifier_valeur` ne change pas `variable` (reste 42).
- `modifier_adresse` change `variable` (devient 999).

Interprétation : Cela prouve concrètement que C passe toujours par valeur (la copie `v` a une adresse différente), mais qu'en passant `&variable`, on donne à la fonction les moyens d'agir sur l'original.

À RETENIR Synthèse : Variables, Types et Adressage

- Types de base : `int`, `float`, `double`, `char`. Déclaration : `type nom = valeur;`
- Mémoire : Chaque variable a une adresse unique (hexadécimal).
- Opérateur `&` : `&x` donne l'adresse de `x`. Indispensable pour `scanf` et le passage par adresse.
- Passage par valeur (défaut) : Copie locale. Ne modifie pas l'original.
- Passage par adresse (simulé) : On passe `&x`. Paramètre formel : pointeur `type *p`. Accès à la valeur : `*p`. Modifie l'original.
- Règle d'or : Pour modifier un argument → passage d'adresse (`&` à l'appel, `*` dans la fonction).

4 LES STRUCTURES RÉPÉTITIVES : FOR, WHILE, DO...WHILE

Dans la section précédente, tu as appris à déclarer des variables, à choisir le bon type (`int`, `float`, `char`) et tu as compris le mode d'adressage qui permet à `scanf` de modifier une variable grâce à l'opérateur `&`. Tu sais donc maintenant stocker et saisir des données. Mais un programme qui ne fait les choses qu'une seule fois reste limité : imagine que le proviseur te demande d'afficher la table de multiplication de 7, ou de calculer la somme des notes de 60 élèves d'une classe de Première C. Vas-tu écrire 60 fois la même instruction ? Bien sûr que non : il faut **répéter** des instructions. C'est le rôle des **structures répétitives** (ou boucles), que tu as déjà rencontrées en algorithmique avec `Pour` et `TantQue`. Nous allons maintenant les traduire en langage C.

4.1 Rappel algorithmique : les deux grandes familles de boucles

En algorithmique, tu as distingué deux situations :

- La **boucle à nombre d'itérations connu** : on sait à l'avance combien de fois on va répéter. Exemple : « afficher la table de 7, de 1 à 10 ». On utilise `Pour ... FinPour`.
- La **boucle à condition d'arrêt** : on ne sait pas combien de fois on va répéter ; on continue tant qu'une condition est vraie. Exemple : « redemander la note tant qu'elle n'est pas entre 0 et 20 ». On utilise `TantQue ... FinTantQue` OU `Répéter ... Jusqu'à`.

Le langage C propose exactement trois instructions pour couvrir ces situations : `for`, `while` et `do...while`. Le tableau de correspondance suivant résume tout.

Algorithmique (pseudo-code)	Langage C
Pour i de 1 à n Faire ... FinPour	> <code>for(i=1; i<=n; i++){ ... }</code>
TantQue condition Faire ... FinTantQue	> <code>while(condition) { ... }</code>
Répéter ... Jusqu'à condition	> <code>do { ... } while(condition);</code>

Correspondance entre les boucles de l'algorithmique et celles du langage C.

ATTENTION Inversion de sens pour `do...while`

Le `Répéter ... Jusqu'à` de l'algorithmique s'arrête quand la condition devient vraie, alors que le `do...while` du C continue tant que la condition est vraie. Les conditions sont donc contraires : « Jusqu'à note valide » devient « `while(note invalide)` ».

4.2 La boucle `for` : quand le nombre d'itérations est connu

DÉFINITION Boucle `for`

La boucle **for** répète un bloc d'instructions un nombre de fois déterminé, en faisant évoluer automatiquement une variable appelée **compteur**. Sa syntaxe est :

```
for( initialisation ; condition ; increment ){
    instructions;
}
```

Les trois parties entre parenthèses, séparées par des **points-virgules**, jouent chacune un rôle précis :

1. **initialisation** : exécutée **une seule fois**, au tout début (ex. `i=1`);
2. **condition** : testée **avant chaque itération**; si elle est fausse, la boucle s'arrête (ex. `i<=10`);
3. **increment** : exécuté **après chaque itération**, pour faire progresser le compteur (ex. `i++`).

Pour écrire ces conditions, tu disposes des opérateurs de comparaison du C : `==` (égal), `!=` (différent), `<`, `>`, `<=`, `>=`. Et pour faire évoluer le compteur, le C offre des opérateurs raccourcis très pratiques :

Opérateur	Équivalent	Effet
<code>i++</code>	<code>i = i + 1</code>	incrémente <code>i</code> de 1
<code>i--</code>	<code>i = i - 1</code>	décrémente <code>i</code> de 1
<code>i += 3</code>	<code>i = i + 3</code>	augmente <code>i</code> de 3
<code>i -= 2</code>	<code>i = i - 2</code>	diminue <code>i</code> de 2

EXEMPLE La table de multiplication de 7

Voici le programme complet qui affiche la table de 7, de $7 \times 1 = 7$ jusqu'à $7 \times 10 = 70$:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;                // compteur de la boucle

    printf("Table de multiplication de 7\n");
    for(i = 1; i <= 10; i++) {
        printf("7 x %d = %d\n", i, 7 * i);
    }
    return 0;
}
```

Déroulement : `i` vaut 1, le test `i<=10` est vrai, on affiche « 7 x 1 = 7 », puis `i++` fait passer `i` à 2, et ainsi de suite. Quand `i` atteint 11, le test devient faux et la boucle s'arrête. Le programme a donc affiché exactement 10 lignes avec seulement 3 lignes de code.

4.3 La boucle while : le test avant chaque itération

DÉFINITION Boucle while

La boucle **while** (« tant que ») répète un bloc d'instructions tant qu'une condition est vraie. Le test est effectué avant chaque itération : si la condition est fausse dès le départ, le corps de la boucle n'est jamais exécuté.

```
while(condition) {
    instructions;
```

```
| }
```

C'est la boucle idéale quand on ne connaît pas le nombre de répétitions à l'avance. Par exemple, calculons la somme $S = 1 + 2 + \dots + n$ où n est saisi par l'utilisateur :

```
#include <stdio.h>

int main() {
    int n, i, somme;

    printf("Entrez n : ");
    scanf("%d", &n);    // ne pas oublier le & !

    i = 1;              // initialisation AVANT la boucle
    somme = 0;
    while(i <= n) {     // test avant chaque iteration
        somme = somme + i; // accumulation
        i++;           // increment OBLIGATOIRE
    }
    printf("Somme = %d\n", somme);
    return 0;
}
```

Pour $n = 4$, la table d'exécution est la suivante :

Itération	Test $i \leq n$	somme	i après $i++$
1	$1 \leq 4$: vrai	1	2
2	$2 \leq 4$: vrai	3	3
3	$3 \leq 4$: vrai	6	4
4	$4 \leq 4$: vrai	10	5
5	$5 \leq 4$: faux, sortie	10	5

Le programme affiche bien `Somme = 10`.

4.4 La boucle `do...while` : le corps s'exécute au moins une fois

DÉFINITION Boucle `do...while`

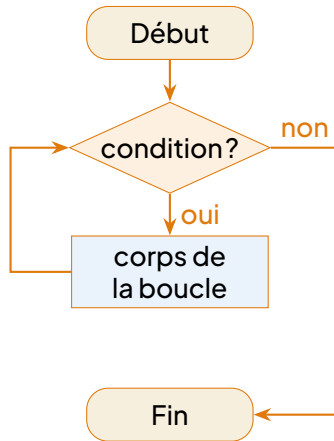
La boucle **do...while** exécute d'abord le corps, puis teste la condition : le corps est donc exécuté au moins une fois, même si la condition est fausse dès le départ.

```
do {
    instructions;
} while(condition);
```

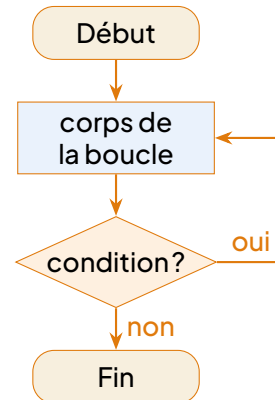
Remarque le point-virgule obligatoire après `while(condition)` : c'est la seule boucle du C qui se termine ainsi.

L'organigramme ci-dessous met en évidence la différence fondamentale entre `while` (test en entrée) et `do...while` (test en sortie).

while : test en entrée



do...while : test en sortie



Comparaison des organigrammes : avec `while`, le test précède le corps ; avec `do...while`, le corps s'exécute au moins une fois avant le premier test.

Application typique : la **saisie contrôlée**. On veut obliger l'utilisateur à entrer une note entre 0 et 20. Il faut lui poser la question au moins une fois, et recommencer tant que la valeur est invalide :

```

#include <stdio.h>

int main() {
    float note;

    do {
        printf("Entrez une note entre 0 et 20 : ");
        scanf("%f", &note);
    } while(note < 0 || note > 20); // on recommence si invalide

    printf("Note validee : %.2f\n", note);
    return 0;
}
  
```

Ici, la condition de poursuite `note < 0 || note > 20` est bien le contraire de la condition d'arrêt « note valide » de l'algorithmique.

4.5 Équivalence entre `for` et `while`

PROPRIÉTÉ Toute boucle `for` peut s'écrire avec `while`

Les deux constructions suivantes sont strictement équivalentes :

```

for(i = 1; i <= n; i++){
    instructions;
  
```

```
| }  
  
| i = 1;           // initialisation sortie de la boucle  
| while(i <= n) { // meme condition  
|     instructions;  
|     i++;        // increment deplace a la fin du corps  
| }
```

Démonstration guidée, ligne à ligne, sur la table de 7. Partons du `for` :

1. On **extraît l'initialisation** `i = 1` et on la place avant la boucle : c'est la première chose que le `for` exécute, une seule fois.
2. La **condition** `i <= 10` devient la condition du `while` : dans les deux cas, elle est testée avant chaque passage.
3. Le corps `printf(...)` reste **inchangé**.
4. L'**incrément** `i++`, que le `for` exécute après chaque passage, est recopié comme dernière instruction du corps du `while`.

On obtient :

```
| i = 1;  
| while(i <= 10) {  
|     printf("7 x %d = %d\n", i, 7 * i);  
|     i++;  
| }
```

Ce programme affiche exactement les mêmes 10 lignes que la version avec `for`. Le `for` n'est donc qu'une écriture plus compacte et plus sûre (on ne risque pas d'oublier l'incrément, il est dans l'en-tête).

4.6 Choisir la bonne boucle et éviter la boucle infinie

MÉTHODE Quelle boucle choisir ?

Face à un problème, pose-toi ces questions dans l'ordre :

1. Le nombre de répétitions est-il connu à l'avance ? (table de multiplication, 60 élèves, 12 mois) ⇒ utilise `for`.
2. La répétition dépend-elle d'une condition, avec un test à faire **AVANT** la première exécution ? (tant que la somme ne dépasse pas un seuil, tant qu'on n'a pas trouvé) ⇒ utilise `while`.
3. Le corps doit-il s'exécuter au moins une fois avant tout test ? (saisie contrôlée, menu) ⇒ utilise `do...while`.

ATTENTION Les deux erreurs qui créent une boucle infinie

Erreur n°1 : le point-virgule juste après le test.

```
while(i <= 10);      // ERREUR : ce ; cree un corps VIDE
{
    printf("%d\n", i);
    i++;
}
```

Le ; seul constitue le corps de la boucle : le programme tourne indéfiniment sur une boucle vide, car *i* ne change jamais. Le bloc entre accolades n'est exécuté qu'après... donc jamais.

Erreur n°2 : oublier l'incrément.

```
i = 1;
while(i <= 10) {
    printf("%d\n", i);
    // i++ oubliée : i reste à 1, boucle infinie !
}
```

La condition reste vraie éternellement. Dans toute boucle `while` ou `do...while`, vérifie toujours qu'au moins une instruction du corps fait progresser la condition vers l'arrêt.

LESAVAIS-TU ? `do...while` : la boucle des menus

Le `do...while` est la boucle préférée des programmeurs pour les menus interactifs. Pourquoi ? Parce qu'un menu doit s'afficher au moins une fois pour que l'utilisateur puisse choisir, puis on recommence tant que le choix est invalide ou différent de « Quitter ». C'est exactement le principe des applications mobiles camerounaises de transfert d'argent : le menu « 1. Envoyer / 2. Retirer / 0. Quitter » s'affiche, puis se réaffiche tant que tu n'as pas choisi 0. En C :

```
int choix;
do{
    printf("1. Envoyer 2. Retirer 0. Quitter : ");
    scanf("%d", &choix);
} while(choix != 0);
```

EXERCICE RÉSOLU 4 Somme des *n* premiers entiers pairs

Énoncé. Écrire un programme C qui demande un entier $n \geq 1$ à l'utilisateur (avec saisie contrôlée), puis calcule et affiche la somme des *n* premiers entiers pairs : $S = 2 + 4 + 6 + \dots + 2n$.

SOLUTION

Solution détaillée. Analysons : la saisie contrôlée exige un `do...while` (on pose la question au moins une fois) ; le calcul de la somme porte sur un nombre connu de termes (n termes), donc un `for`. Le i -ème entier pair vaut $2i$.

```
#include <stdio.h>

int main() {
    int n, i, somme = 0;

    // Saisie contrôlée : n doit être >= 1
    do {
        printf("Entrez un entier n >= 1 : ");
        scanf("%d", &n);
    } while(n < 1);

    // Somme des n premiers pairs : 2 + 4 + ... + 2n
    for(i = 1; i <= n; i++) {
        somme += 2 * i; // équivalent à somme = somme + 2*i
    }

    printf("Somme des %d premiers pairs = %d\n", n, somme);
    return 0;
}
```

Vérification pour $n = 3$: la boucle ajoute 2, puis 4, puis 6, donc `somme = 12`. Le programme affiche : Somme des 3 premiers pairs = 12. C'est correct, car $2 + 4 + 6 = 12$ (on retrouve d'ailleurs la formule $S = n(n + 1) = 3 \times 4 = 12$).

EXERCICE 3 À toi de jouer

1. Écris un programme qui affiche, à l'aide d'une boucle `for`, les carrés des entiers de 1 à 12 (1, 4, 9, ..., 144).
2. Réécris le programme de la table de 7 en utilisant uniquement une boucle `while`.
3. Écris un programme qui demande un mot de passe (entier, par exemple 1234) et recommence la saisie tant que le mot de passe est faux. Quelle boucle choisis-tu et pourquoi?

CORRIGÉ Exercice

```
int i;
for(i = 1; i <= 12; i++) {
    printf("%d\n", i * i);
}
```

```
| }
```

```
int i = 1;
while(i <= 10) {
    printf("7 x %d = %d\n", i, 7 * i);
    i++;
}
```

3. On choisit `do...while`, car il faut demander le mot de passe au moins une fois avant de pouvoir le tester :

```
int mdp;
do{
    printf("Mot de passe :");
    scanf("%d", &mdp);
} while(mdp != 1234);
printf("Acces autorise\n");
```

À RETENIR L'essentiel des boucles en C

- `for(init ; condition ; increment)` : nombre d'itérations connu ; l'en-tête regroupe tout le pilotage de la boucle.
- `while(condition)` : test avant chaque passage ; le corps peut ne jamais s'exécuter.
- `do {...} while(condition)` : test après ; le corps s'exécute au moins une fois ; attention au ; final.
- Tout `for` se réécrit en `while` : initialisation avant, condition dans le test, incrément à la fin du corps.
- Jamais de ; juste après `while(condition)`, et jamais de boucle sans instruction qui fait progresser la condition vers l'arrêt.

5 LES TABLEAUX À UNE DIMENSION

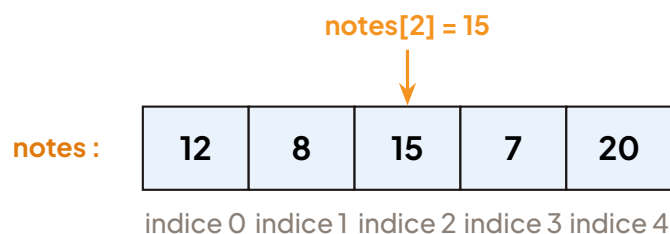
Dans la section précédente, tu as appris à répéter des traitements grâce aux boucles `for`, `while` et `do...while`. Ces boucles deviennent véritablement puissantes lorsqu'elles manipulent des collections de données. Imagine le professeur de Première C du lycée de Nkongsamba qui veut calculer la moyenne des notes d'informatique de ses 5 élèves. Jusqu'ici, il faudrait déclarer cinq variables distinctes : `note1`, `note2`, `note3`, `note4`, `note5`. Et s'il y avait 60 élèves ? Soixante variables, soixante `scanf`, soixante additions écrites à la main : c'est impraticable. Le langage C résout ce problème avec une structure fondamentale : le **tableau**.

5.1 Définition et intuition

DÉFINITION Tableau à une dimension

Un **tableau à une dimension** est une **variable composée** qui regroupe, sous un seul nom, plusieurs valeurs de même type, stockées dans des cases mémoire contiguës. Chaque case est repérée par un numéro appelé **indice**, qui commence toujours à 0.

L'image mentale à retenir est celle d'une rangée de casiers numérotés dans la salle des professeurs : tous les casiers ont la même taille (même type de contenu), ils sont placés les uns à côté des autres (cases contiguës), et on accède à chacun par son numéro. La seule surprise pour un débutant : le premier casier porte le numéro 0, pas le numéro 1.



Représentation mémoire du tableau `notes[5]` : cinq cases contiguës indicées de 0 à 4. L'accès `notes[2]` désigne la troisième case, qui contient la valeur 15.

À RETENIR Les trois caractéristiques d'un tableau

- Un seul nom pour toute la collection (ici `notes`);
- un seul type pour toutes les cases (un tableau de `float` ne contient que des `float`);
- des indices de 0 à `taille - 1` : un tableau de 5 cases a les indices 0, 1, 2, 3 et 4.

5.2 Déclaration d'un tableau

PROPRIÉTÉ Syntaxe de déclaration

La déclaration d'un tableau suit la syntaxe :

```
type nom[taille];
```

où `type` est le type commun des éléments (`int`, `float`, `char`...), `nom` est l'identificateur du tableau et `taille` est un entier constant indiquant le nombre de cases. La déclaration ré-

serve en mémoire `taille` cases contiguës.

Par exemple, pour stocker les notes de 5 élèves :

```
float notes[5]; /* reserve 5 cases de type float, indices 0 a 4 */
int ages[10]; /* reserve 10 cases de type int, indices 0 a 9 */
char nom[20]; /* reserve 20 cases de type char (chaine de caracteres) */
```

ATTENTION Taille fixée à la déclaration

La taille d'un tableau est fixée une fois pour toutes à la déclaration : on ne peut pas l'agrandir ensuite. Choisis donc une taille suffisante dès le départ (par exemple `float notes[60]` ; pour une classe complète, quitte à n'utiliser qu'une partie des cases).

5.3 Accès aux éléments et initialisation

Pour lire ou modifier une case, on écrit le nom du tableau suivi de l'indice entre crochets :

```
float notes[5];
notes[0] = 12.5; /* ecrit 12.5 dans la premiere case */
notes[4] = 18.0; /* ecrit 18.0 dans la derniere case */
printf("La premiere note est %.2f\n", notes[0]);
```

L'indice peut être n'importe quelle expression entière : une constante (`notes[2]`), une variable (`notes[i]`) ou même un calcul (`notes[i+1]`). C'est cette souplesse qui rend les boucles si efficaces avec les tableaux.

PROPRIÉTÉ Initialisation à la déclaration

On peut donner les valeurs directement à la déclaration, entre accolades, séparées par des virgules :

```
int t[5] = {12, 8, 15, 7, 20};
```

Les valeurs sont placées dans l'ordre : `t[0]` vaut 12, `t[1]` vaut 8, ..., `t[4]` vaut 20. Si l'on fournit moins de valeurs que la taille, les cases restantes sont mises à 0.

ATTENTION Le premier élément est `t[0]`, pas `t[1]`

L'erreur la plus fréquente chez les débutants est de croire que `t[1]` est la première case. Non : `t[0]` est la première, `t[1]` la deuxième, et pour un tableau de taille `n`, la dernière case est `t[n-1]`. Accéder à `t[n]` (par exemple `notes[5]` pour un tableau de taille 5) est un **dé-**

passement d'indice : le C ne signale aucune erreur à la compilation, mais le programme lit ou écrit en dehors de la zone réservée, ce qui provoque un comportement imprévisible (valeurs absurdes, plantage, résultats faux).

5.4 Remplissage d'un tableau avec une boucle for et scanf

Saisir les cases une par une serait absurde : c'est exactement le travail d'une boucle `for`. La variable de boucle joue le rôle d'indice.

MÉTHODE Remplir un tableau de taille `n`

1. Déclarer le tableau : `float notes[5];`
2. Déclarer une variable entière d'indice : `int i;`
3. Faire varier `i` de 0 à `n-1` avec `for(i = 0; i < n; i++);`
4. Dans le corps de la boucle, saisir la case courante : `scanf("%f", ¬es[i]);`

Remarque bien la présence du `&` devant `notes[i]` : chaque case du tableau est une variable ordinaire, et `scanf` a besoin de son adresse (c'est le mode d'adressage vu dans la section 3).

```
int i;
float notes[5];

for (i = 0; i < 5; i++){
    printf("Entrez la note de l'élève %d : ", i + 1);
    scanf("%f", &notes[i]); /* & obligatoire : case = variable float */
}
```

À l'exécution, l'utilisateur saisit successivement `notes[0]`, puis `notes[1]`, ..., jusqu'à `notes[4]`. Le message affiche `i + 1` pour que l'utilisateur voie « élève 1 » à « élève 5 », même si les indices internes vont de 0 à 4.

5.5 Parcours et traitements classiques

Une fois le tableau rempli, on le **parcourt** — c'est-à-dire qu'on visite chaque case une à une — pour y appliquer un traitement. Tous les parcours classiques reposent sur le même squelette de boucle.

MÉTHODE Le squelette universel du parcours

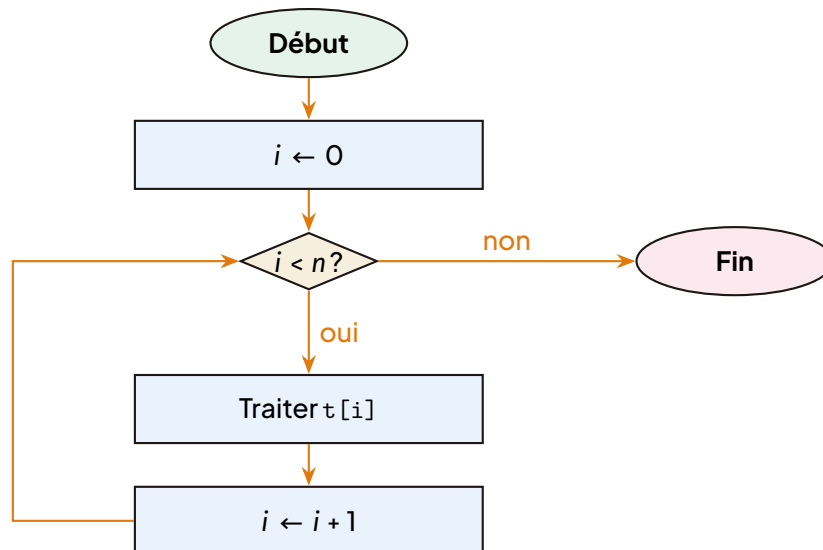
Pour traiter un tableau de taille `n`, utilise systématiquement :

```
for (i = 0; i < n; i++){
    /* traitement de t[i] */
}
```

```
| }

```

La condition est $i < n$, jamais $i \leq n$. Avec $i \leq n$, la boucle atteindrait $i == n$, donc la case $t[n]$ qui n'existe pas : dépassement d'indice garanti.



Organigramme du parcours d'un tableau par une boucle `for` : l'indice i part de 0, le corps traite la case $t[i]$, puis i est incrémenté tant que $i < n$.

Voyons les quatre traitements que tu dois savoir écrire les yeux fermés.

1. Affichage de tous les éléments.

```
for (i = 0; i < 5; i++){
    printf("notes[%d] = %.2f\n", i, notes[i]);
}
```

2. Somme des éléments. On utilise un accumulateur initialisé à 0, auquel on ajoute chaque case :

```
float somme = 0;
for (i = 0; i < 5; i++){
    somme = somme + notes[i]; /* accumulation */
}
```

3. Moyenne. C'est la somme divisée par le nombre d'éléments :

```
float moyenne = somme / 5;
```

4. Recherche du maximum. On suppose que le maximum est la première case, puis on le corrige chaque fois qu'on rencontre une case plus grande :

```
float max = notes[0]; /* hypothese de depart */
for (i = 1; i < 5; i++){ /* on commence à l'indice 1 */
```

```
if (notes[i] > max){
    max = notes[i];    /* nouveau champion */
}
}
printf("La meilleure note est %.2f\n", max);
```

ATTENTION Initialiser avant la boucle

Un accumulateur (`somme`) ou un champion (`max`) doit être initialisé avant la boucle, jamais dedans. Si tu écris `somme = 0;` dans le corps de la boucle, elle est remise à zéro à chaque tour et le résultat final ne contient que la dernière case.

LESAVAIS-TU ? Le nom d'un tableau est une adresse

En C, le nom d'un tableau représente l'adresse de sa première case. C'est pourquoi, pour une chaîne de caractères, on écrit `scanf("%s", nom);` sans le symbole `&` : `nom` est déjà une adresse. En revanche, pour une case individuelle comme `notes[i]` (qui est une simple valeur `float`), le `&` reste obligatoire. Cette subtilité explique aussi pourquoi les tableaux se transmettent si facilement aux fonctions : on passe en réalité l'adresse du début du tableau.

5.6 Exemple résolu : la moyenne de la classe

EXERCICE RÉSOLU 5 Moyenne des notes de 5 élèves

Énoncé. Écris un programme C complet qui saisit les notes d'informatique de 5 élèves dans un tableau, puis calcule et affiche la somme, la moyenne de la classe et la meilleure note.

SOLUTION

Solution détaillée. On applique la méthode : déclaration, remplissage par boucle, puis trois parcours (somme, affichage, maximum). On inclut `conio.h` pour `getch()` comme le prévoit le programme officiel.

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>

int main() {
    float notes[5];    /* tableau des 5 notes */
    float somme = 0;    /* accumulateur */
    float moyenne, max;
    int i;

    /* 1. Remplissage du tableau */
```

```
for (i = 0; i < 5; i++){
    printf("Entrez la note de l'eleve %d :", i + 1);
    scanf("%f", &notes[i]);
}

/* 2. Calcul de la somme */
for (i = 0; i < 5; i++){
    somme = somme + notes[i];
}
moyenne = somme / 5;

/* 3. Recherche du maximum */
max = notes[0];
for (i = 1; i < 5; i++){
    if (notes[i] > max){
        max = notes[i];
    }
}

/* 4. Affichage des resultats */
printf("\nSomme des notes : %.2f\n", somme);
printf("Moyenne de classe : %.2f\n", moyenne);
printf("Meilleure note : %.2f\n", max);

getch();
return 0;
}
```

Exemple d'exécution avec les notes 12, 8, 15, 7 et 20 : la somme vaut $12 + 8 + 15 + 7 + 20 = 62$, la moyenne vaut $62 / 5 = 12,40$ et la meilleure note est 20. Le programme affiche donc :

```
Somme des notes : 62.00
Moyenne de classe : 12.40
Meilleure note : 20.00
```

5.7 Erreurs fréquentes et bonnes habitudes

ATTENTION Les quatre pièges classiques des tableaux

- **Décalage d'indice** : écrire `for(i = 1; i <= 5; i++)` au lieu de `for(i = 0; i < 5; i++)`. La boucle saute la case 0 et touche la case 5 qui n'existe pas.
- **Condition `i <= n`** : avec une taille `n`, le dernier indice valide est `n - 1`. La condition doit être `i < n`.
- **Oublier le `&` dans `scanf`** : `scanf("%f", notes[i]);` sans `&` provoque un plantage, car `scanf` attend une adresse.
- **Tableau non initialisé** : lire une case avant de l'avoir remplie donne une valeur aléatoire (« résidu » de la mémoire). Remplis toujours le tableau avant de le parcourir.

5.8 Exercices d'application

EXERCICE 4 À toi de jouer

1. Déclare un tableau `age` pouvant contenir les âges de 10 élèves, puis écris la boucle qui les saisit au clavier.
2. Écris un programme qui remplit un tableau de 6 entiers et affiche le minimum du tableau.
3. Écris un programme qui saisit 8 températures relevées à Douala et affiche celles qui dépassent 30 °C.
4. Que se passe-t-il si, pour `int t[5];`, on exécute `t[5] = 100;` ? Explique pourquoi c'est dangereux.

CORRIGÉ Exercice 2

```
#include <stdio.h>

int main() {
    int t[6];
    int i, min;

    for (i = 0; i < 6; i++) {
        printf("Entrez l'entier %d : ", i + 1);
        scanf("%d", &t[i]);
    }

    min = t[0];           /* hypothese de depart */
    for (i = 1; i < 6; i++) {
        if (t[i] < min) {
            min = t[i];
        }
    }
    printf("Le minimum est %d\n", min);
    return 0;
}
```

Le raisonnement est symétrique de celui du maximum : on part de la première case et l'on corrige le champion dès qu'une case plus petite apparaît.

À RETENIR L'essentiel sur les tableaux à une dimension

- Un tableau regroupe sous un seul nom plusieurs valeurs de même type dans des cases contiguës.

- **Déclaration** : `type nom[taille];` — les indices vont de 0 à `taille - 1`.
- **Initialisation** : `int t[5] = {12, 8, 15, 7, 20};`
- **Tout traitement** (saisie, affichage, somme, moyenne, maximum) s'écrit avec `for(i = 0; i < n; i++)`.
- **Le nom d'un tableau** est l'adresse de sa première case ; une case individuelle reste une variable ordinaire qui exige `&` dans `scanf`.

Tu sais désormais stocker et traiter des collections de données : c'est un cap décisif. Dans la prochaine section, nous verrons comment compiler, tester et corriger tes programmes dans l'IDE, car un programme qui manipule des tableaux doit être vérifié avec soin, notamment aux frontières des indices.

6 COMPILATION, TESTS ET CORRECTION D'UN PROGRAMME

Dans la section précédente, nous avons appris à déclarer et à manipuler des **tableaux** à une dimension, par exemple pour stocker les notes de toute une classe. Mais un programme, même bien écrit sur le papier, ne fonctionne presque jamais du premier coup : il faut le **compiler**, lire les messages du compilateur, **corriger** les erreurs, puis le **tester** méthodiquement. C'est cette dernière étape du travail du programmeur — souvent la plus longue dans la pratique — que nous allons maintenant maîtriser. Imagine un professeur du lycée de Nkongsamba qui veut calculer automatiquement la moyenne de sa classe de Première D : un seul caractère mal placé dans son programme peut fausser les moyennes de 80 élèves !

6.1 Rôle du compilateur

DÉFINITION Compilateur

Le **compilateur** est un programme qui traduit le **code source** (fichier `.c` écrit par le programmeur) en **langage machine** exécutable par le processeur (fichier `.exe` sous Windows). Avant de traduire, il effectue une **vérification de la syntaxe** : chaque instruction est-elle bien formée ? chaque variable est-elle déclarée ? les parenthèses et accolades sont-elles équilibrées ?

Le compilateur C (par exemple `gcc`, intégré aux IDE comme Code :: Blocks ou Dev-C++) travaille en deux temps principaux :

1. **Analyse (vérification syntaxique)** : il lit le fichier source ligne par ligne et vérifie la conformité aux règles du langage C. S'il détecte une erreur, il s'arrête et affiche un message indiquant le fichier, le numéro de ligne et la nature du problème.

2. **Traduction (génération de code)** : si aucune erreur bloquante n'est détectée, il produit le programme exécutable en langage machine (souvent via une étape d'édition de liens pour inclure les bibliothèques).

PROPRIÉTÉ Erreurs et avertissements

Le compilateur distingue deux catégories de messages :

- les **erreurs** (error) : elles bloquent la compilation ; aucun exécutable n'est produit tant qu'elles ne sont pas corrigées ;
- les **avertissements** (warning) : ils ne bloquent pas la compilation, mais signalent une construction suspecte (variable non initialisée, format douteux dans `printf`, conversion implicite...). Un avertissement est souvent le signe d'un futur bug : il faut toujours les lire et les traiter.

MÉTHODE Comment lire les messages du compilateur

1. Compile le programme (touche F9 dans Code ::Blocks, F11 dans Dev-C++).
2. Lis le premier message d'erreur de la liste, pas le dernier : une seule erreur (par exemple un ; oublié) peut en déclencher plusieurs autres en cascade sur les lignes suivantes.
3. Double-clique sur le message : l'IDE place le curseur sur la ligne fautive (ou juste après).
4. Corrige, puis recompile immédiatement. Souvent, plusieurs messages disparaissent d'un coup.
5. Recommence jusqu'à obtenir « 0 erreur, 0 avertissement ».

6.2 Les erreurs typiques du débutant en C

Erreur	Message typique du compilateur	Correction
Point-virgule manquant	error: expected ';' before ...	Ajouter ; à la fin de l'instruction précédente.
Variable non déclarée	error: 'note' undeclared (first use in this function)	Déclarer la variable avant usage : <code>float note;</code>
Format % incorrect	warning: format '%d' expects argument of type 'int*'...	%d pour int, %f pour float, %lf pour double.
& oublié dans scanf	Pas d'erreur à la compilation, mais plantage (segmentation fault) à l'exécution	<code>scanf("%f", &note);</code> : & donne l'adresse de la variable.
Accolade manquante	error: expected '}' at end of input	Vérifier que chaque { a son } (indentation rigoureuse).

ATTENTION Compiler ne suffit pas !

Un programme qui compile sans erreur n'est pas forcément juste. Le compilateur vérifie la syntaxe, pas la logique (sémantique). Par exemple, la boucle `for(i = 1; i <= n; i++)` compile parfaitement, mais si le tableau est déclaré `float notes[30]` (indices 0 à 29), l'indice `n` peut dépasser la taille du tableau ou fausser le calcul. Seuls les tests valident la logique du programme.

6.3 Méthode de test : le jeu d'essai

DÉFINITION Jeu d'essai

Un **jeu d'essai** (ou jeu de test) est un ensemble de valeurs d'entrée choisies délibérément, accompagné pour chacune du résultat attendu, calculé à la main avant l'exécution. On compare ensuite le résultat affiché par le programme au résultat attendu.

MÉTHODE Construire un bon jeu d'essai

Pour tester un programme, prévois au moins trois catégories de valeurs :

1. les valeurs ordinaires : celles du cas normal (par exemple des notes 12, 14, 9) ;
2. les valeurs limites : celles qui se situent aux bornes du domaine (0 et 20 pour une note) ;
3. les valeurs aberrantes (invalides) : celles que le programme devrait rejeter ou traiter avec prudence (-5, 25 pour une note).

Pour chaque cas, note dans un tableau : les entrées, le résultat attendu, le résultat obtenu,

et ta conclusion (test réussi ou échoué).

N°	Entrées (notes)	Catégorie	Résultat attendu	Verdict
1	12; 14; 9	ordinaire	moyenne = 11,67	à vérifier
2	0; 0	limite basse	moyenne = 0,00	à vérifier
3	20; 20	limite haute	moyenne = 20,00	à vérifier
4	-5; 10	aberrante	rejet ou message d'erreur	à vérifier
5	25; 15	aberrante	rejet ou message d'erreur	à vérifier

Jeu d'essai préparé pour le programme « moyenne d'une classe ».

6.4 Débogage : suivre l'exécution pas à pas

Lorsqu'un programme compile mais donne un résultat faux, il faut observer ce qui se passe à l'intérieur. Deux outils simples suffisent à notre niveau :

- le **printf de débogage** : on insère temporairement des affichages des valeurs des variables à des points stratégiques (dans une boucle, après un calcul) ;
- `getch()` : en plaçant cette fonction (bibliothèque `conio.h`, spécifique aux environnements Windows comme Code ::Blocks/Dev-C++) à des endroits choisis, on **met le programme en pause** ; il attend qu'on appuie sur une touche, ce qui laisse le temps de lire les affichages.

EXEMPLE Trace d'exécution avec printf et getch

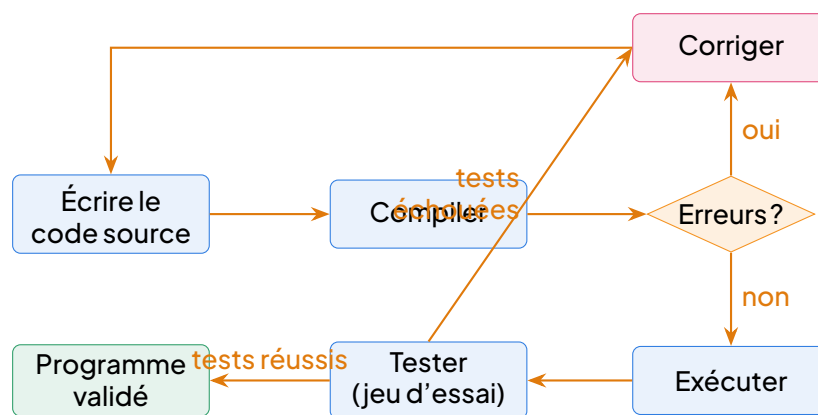
```
for (i = 0; i < n; i++){
    somme = somme + notes[i];
    /* Affichage temporaire de débogage */
    printf("DEBUG : i = %d, note = %.2f, somme = %.2f\n",
        i, notes[i], somme);
    getch(); /* pause : appuyer sur une touche pour continuer */
}
```

Une fois l'erreur trouvée et corrigée, on retire ces instructions de débogage.

6.5 La démarche complète de développement

MÉTHODE Du problème au programme fiable

1. Analyser le problème : identifier les entrées, les traitements, les sorties.
2. Écrire l'algorithme en pseudo-code, sur papier.
3. Coder en C dans l'IDE, en respectant la structure du programme.
4. Compiler et corriger les erreurs de syntaxe, en commençant par la première.
5. Tester avec le jeu d'essai préparé (valeurs ordinaires, limites, aberrantes).
6. Corriger les erreurs de logique révélées par les tests, puis recommencer le cycle.



Le cycle de développement : on tourne dans la boucle « écrire – compiler – corriger » jusqu'à la validation par les tests.

6.6 Correction guidée d'un programme fautif

EXERCICE RÉSOLU 6 Quatre erreurs à traquer

Voici un programme censé calculer la moyenne de n notes saisies au clavier. Il contient 4 erreurs types. Trouve-les et corrige-les.

```
#include <stdio.h>
int main() {
    int n, i;
    float note, somme, moyenne;
    printf("Nombre de notes : ");
    scanf("%d", n);
    somme = 0
    for (i = 0; i <= n; i++) {
        printf("Note %d : ", i + 1);
        scanf("%f", &note);
        somme = somme + note;
```

```

    }
    moyenne = somme / n;
    printf("Moyenne = %d\n", moyenne);
    return 0;
}

```

SOLUTION

Correction ligne par ligne :

1. `scanf("%d", n);` → il manque l'adresse de la variable : `scanf("%d", &n);` (le caractère `&` donne l'adresse mémoire de `n`).
2. `somme = 0` → point-virgule manquant : `somme = 0;`
3. `for (i = 0; i <= n; i++)` → la boucle s'exécute $n + 1$ fois au lieu de n : il faut `i < n`. C'est une erreur de logique : le compilateur ne la signale pas, seuls les tests la révèlent !
4. `printf("Moyenne = %d\n", moyenne);` → `moyenne` est un `float` : il faut le format `%f` (idéalement `%.2f` pour deux décimales).

Programme corrigé :

```

#include <stdio.h>
int main() {
    int n, i;
    float note, somme, moyenne;
    printf("Nombre de notes : ");
    scanf("%d", &n);          /* erreur 1 : & ajouté */
    somme = 0;                /* erreur 2 : ; ajouté */
    for (i = 0; i < n; i++) {  /* erreur 3 : < au lieu de <= */
        printf("Note %d : ", i + 1);
        scanf("%f", &note);
        somme = somme + note;
    }
    moyenne = somme / n;
    printf("Moyenne = %.2f\n", moyenne); /* erreur 4 : %f */
    return 0;
}

```

6.7 Atelier guidé : le programme « moyenne d'une classe »

EXPÉRIENCE TP — Écrire, compiler, tester et valider un programme complet

Objectif : produire un programme fiable qui calcule la moyenne des notes d'une classe, en refusant les notes hors de l'intervalle $[0; 20]$.

Matériel : un ordinateur avec un IDE C installé (Code ::Blocks ou Dev-C++).

Étape 1 — Analyse. Entrées : le nombre n d'élèves, puis n notes. Traitement : somme puis division par n , avec contrôle de validité de chaque note. Sortie : la moyenne arrondie à

deux décimales.

Étape 2 — Algorithme (extrait).

```
Algorithme MoyenneClasse
Variables : n, i : entier ; notes : tableau reel ; somme, moyenne : reel
Debut
  Lire n
  somme ← 0
  Pour i de 0 à n-1 Faire
    Repeter Lire notes[i] Jusqu'à 0 ≤ notes[i] ≤ 20
    somme ← somme + notes[i]
  FinPour
  moyenne ← somme / n
  Ecrire moyenne
Fin
```

Étape 3 — Codage en C.

```
#include <stdio.h>
#include <conio.h>

int main() {
  int n, i;
  float notes[100]; /* tableau des notes (100 élèves max) */
  float somme, moyenne;

  printf("Nombre d'élèves (max 100) : ");
  scanf("%d", &n);

  /* Protection contre division par zéro et débordement */
  if (n <= 0 || n > 100) {
    printf("Effectif invalide.\n");
    getch();
    return 1;
  }

  somme = 0;
  for (i = 0; i < n; i++) {
    do {
      printf("Note de l'élève %d (0 à 20) : ", i + 1);
      scanf("%f", &notes[i]);
      if (notes[i] < 0 || notes[i] > 20) {
        printf("Note invalide, recommencez !\n");
      }
    } while (notes[i] < 0 || notes[i] > 20);
    somme = somme + notes[i];
  }

  moyenne = somme / n;
  printf("Moyenne de la classe : %.2f / 20\n", moyenne);

  getch(); /* pause pour lire le résultat */
  return 0;
}
```

Étape 4 — Compilation. Appuie sur F9 (Code ::Blocks). Objectif : « 0 error, 0 warning ». Si

un message apparaît, lis le premier, corrige, recompile.

Étape 5 — Tests avec le jeu d'essai. Exécute le programme avec chaque ligne du tableau de jeu d'essai vu plus haut : (12; 14; 9) doit donner 11,67 ; (0; 0) donne 0,00 ; (20; 20) donne 20,00 ; la saisie de -5 ou 25 doit être refusée avec le message « Note invalide ».

Étape 6 — Interprétation. Si tous les tests passent, le programme est validé. Sinon, place des `printf` de débogage dans la boucle pour observer `somme` et `i`, corrige, et recommence le cycle.

ATTENTION Pièges classiques lors des tests

- Tester uniquement avec des valeurs « gentilles » : c'est avec les valeurs limites et aberrantes qu'on découvre les vrais bugs.
- Oublier de retester après une correction : une correction peut introduire un nouveau bug (on parle de régression).
- Laisser les `printf` de débogage dans la version finale du programme.
- Débordement de tableau : dans le TP, si l'utilisateur entre $n > 100$, le programme écrit hors de `notes[100]`. Toujours vérifier les bornes après une saisie de taille.
- Division par zéro : si $n = 0$, l'instruction `moyenne = somme / n` plante le programme. Tester $n > 0$ avant de diviser.
- `conio.h` et `getch()` sont spécifiques à Windows (Code::Blocks, Dev-C++). Sur Linux/macOS, on utilise `getchar()` (dans `stdio.h`) pour faire une pause.

À RETENIR L'essentiel de la section

- Le **compilateur** vérifie la syntaxe puis traduit le code source en langage machine ; les **erreurs** bloquent, les **avertissements** avertissent.
- On corrige toujours la première erreur affichée, puis on recompile.
- Un programme qui compile n'est pas forcément juste : seul un **jeu d'essai** (valeurs ordinaires, limites, aberrantes) valide la logique.
- `printf` de débogage et `getch()` permettent de suivre l'exécution pas à pas.
- La démarche complète : analyser → algorithmiser → coder → compiler → tester → corriger.

LESAVAIS-TU ? D'où vient le mot « bug » ?

On appelle **bug** (littéralement « insecte » en anglais) une erreur de programme, et **débugage** l'action de la corriger. Le terme est devenu célèbre en 1947, lorsque l'équipe de Grace Hopper, pionnière de la programmation, découvrit un véritable insecte (un papillon de nuit) coincé dans un relais de l'ordinateur Harvard Mark II, provoquant une panne. L'insecte fut collé dans le journal de bord avec la mention « premier cas réel de bug trouvé » ! Depuis, « chasser les bugs » est le quotidien de tous les programmeurs du monde — y compris ceux des entreprises technologiques camerounaises qui développent les applications mobiles de paiement que tu utilises peut-être déjà.

EXERCICE 5 À toi de jouer

Écris un programme C qui lit les âges de 10 élèves dans un tableau et affiche l'âge du plus jeune. Prépare un jeu d'essai complet (valeurs ordinaires, limites, aberrantes comme -3 ou 150), compile, teste et rédige ton tableau de résultats.

7 ACTIVITÉ D'INTÉGRATION

7.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de mobiliser simultanément l'ensemble des savoir-faire de la leçon dans un programme C complet et fonctionnel :

- écrire la structure d'un programme C dans un IDE (Code ::Blocks ou Dev-C++) et inclure les bibliothèques nécessaires (`stdio.h`, `stdlib.h`, `conio.h`);
- utiliser les fonctions d'entrée/sortie `printf`, `scanf` et `getch`, en maîtrisant les caractères % (formats) et & (adressage);
- déclarer et exploiter un tableau à une dimension;
- mettre en œuvre une saisie contrôlée avec la boucle `do...while` et un parcours de tableau avec la boucle `for`;
- compiler, corriger les erreurs signalées par l'IDE et valider le programme par un jeu d'essai.

7.2 Situation

Le Lycée Bilingue de Nkongsamba vient d'équiper sa salle informatique. Mme Etoundi, enseignante de Première C et responsable des moyennes, perd chaque trimestre plusieurs heures à calculer à la main les statistiques de sa classe : moyenne générale, note la plus élevée, note la plus faible et nombre d'élèves admis (note supérieure ou égale à 10/20). Elle fait appel à toi et à ton groupe pour automatiser cette tâche.

Elle te fournit le cahier des charges suivant :

1. Le programme demande d'abord le **nombre d'élèves** de la classe. Ce nombre doit être compris entre 1 et 100 ; toute autre valeur est refusée et la question est posée.
2. Le programme saisit ensuite, dans un **tableau**, la note de chaque élève. Une note doit appartenir à l'intervalle [0 ; 20] ; toute note invalide (par exemple -3 ou 25) est refusée et res-saisie.
3. Le programme calcule et affiche : la **moyenne** de la classe (avec deux chiffres après la virgule), la **note maximale**, la **note minimale** et le **nombre d'admis** (note ≥ 10).

Jeu d'essai imposé (5 élèves) : 12 ; 0 ; 20 ; 7,5 ; une note invalide 25 (qui doit être refusée) remplacée par 14. Résultats attendus : moyenne = $\frac{12 + 0 + 20 + 7,5 + 14}{5} = 10,70$; maximum = 20 ; minimum = 0 ; admis = 3.

7.3 Consignes

Par groupes de 3 ou 4, vous réaliserez les tâches suivantes, dans l'ordre :

1. **Conception sur papier.** Écrivez l'algorithme complet en pseudo-code (déclarations, saisies contrôlées, calculs, affichages). Identifiez les variables nécessaires et leurs types.

2. **Choix des boucles.** Justifiez le choix de `do...while` pour les saisies contrôlées et de `for` pour le parcours du tableau.
3. **Codage.** Saisissez le programme dans l'IDE en respectant la structure d'un programme C (bibliothèques, `main`, accolades, points-virgules).
4. **Compilation et correction.** Compilez. Notez chaque message d'erreur affiché par l'IDE, expliquez sa cause et corrigez-le.
5. **Validation.** Exécutez le programme avec le jeu d'essai imposé et vérifiez que les résultats affichés correspondent aux résultats attendus. Testez aussi le cas limite : un seul élève de note 10.
6. **Bilan collectif.** Comparez avec les autres groupes : types choisis (`int` ou `float` pour les notes?), initialisation du maximum et du minimum, clarté des messages affichés.

7.4 Ressources à mobiliser

À RETENIR

Boîte à outils de l'activité

- **Structure minimale :** `#include <stdio.h>`, fonction `int main(void)` ou `void main()`, corps entre accolades, chaque instruction terminée par `;`.
- **Entrées/sorties :** `printf("format", variables)` avec `%d` (entier), `%f` (réel), `%.2f` (réel à 2 décimales); `scanf("%d", &variable)` — ne jamais oublier le `&` (adressage).
- **Saisie contrôlée :** la boucle `do...while` exécute le corps au moins une fois puis recommence tant que la condition de rejet est vraie.
- **Parcours de tableau :** `for (i = 0; i < n; i++)`; les indices vont de 0 à $n - 1$.
- **Moyenne :** $\bar{x} = \frac{\text{somme des notes}}{\text{nombre d'élèves}}$; attention à la division entière en C.

7.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 7

Le bulletin automatique du Lycée de Nkongsamba

Écrire l'algorithme puis le programme C répondant au cahier des charges de Mme Etoundi, et le valider avec le jeu d'essai imposé.

SOLUTION

Étape 1 — Analyse et choix des variables. Il faut : un entier `n` (nombre d'élèves, entre 1 et 100), un tableau `notes` de réels (les notes peuvent être décimales, comme 7,5), un indice entier `i`, un réel `somme`, deux réels `max` et `min`, un entier `admis`. On dimensionne le tableau à 100 cases, borne maximale du cahier des charges.

Étape 2 — Choix des boucles. Pour les saisies contrôlées, on choisit `do...while` car la question doit être posée au moins une fois avant de pouvoir tester la réponse : c'est exactement la sémantique de cette boucle. Pour le parcours du tableau, le nombre de tours est connu (n) : la boucle `for` est la plus naturelle.

Étape 3 — Algorithme en pseudo-code.

```

Algorithme Bulletin
Variables : n, i, admis : Entier
           notes : Tableau[1..100] de Reel
           somme, max, min : Reel
Debut
  Repeter
    Ecrire("Nombre d'eleves (1 a 100) : ")
    Lire(n)
  Jusqu'a (n >= 1) ET (n <= 100)
  Pour i de 1 a n Faire
    Repeter
      Ecrire("Note de l'eleve ", i, " : ")
      Lire(notes[i])
    Jusqu'a (notes[i] >= 0) ET (notes[i] <= 20)
  FinPour
  somme <- 0 ; max <- notes[1] ; min <- notes[1] ; admis <- 0
  Pour i de 1 a n Faire
    somme <- somme + notes[i]
    Si notes[i] > max Alors max <- notes[i] FinSi
    Si notes[i] < min Alors min <- notes[i] FinSi
    Si notes[i] >= 10 Alors admis <- admis + 1 FinSi
  FinPour
  Ecrire("Moyenne : ", somme / n)
  Ecrire("Maximum : ", max, " Minimum : ", min)
  Ecrire("Admis : ", admis)
Fin
  
```

Étape 4 — Traduction en langage C.

```

#include <stdio.h> /* printf, scanf */
#include <stdlib.h> /* fonctions utilitaires */
#include <conio.h> /* getch */

int main(void)
{
    int n, i, admis;
    float notes[100]; /* tableau de 100 cases : indices 0 a 99 */
    float somme, max, min;

    /* 1) Saisie controlee du nombre d'eleves */
    do {
        printf("Nombre d'eleves de la classe (1 a 100) : ");
        scanf("%d", &n); /* & obligatoire : adresse de n */
    } while (n < 1 || n > 100);

    /* 2) Saisie controlee des notes dans le tableau */
    for (i = 0; i < n; i++) {
        do {
            printf("Note de l'eleve %d (0 a 20) : ", i + 1);
            scanf("%f", &notes[i]);
        } while (notes[i] < 0 || notes[i] > 20);
        somme += notes[i];
        if (notes[i] > max) max = notes[i];
        if (notes[i] < min) min = notes[i];
        if (notes[i] >= 10) admis++;
    }
    printf("Moyenne : %.2f\n", somme / n);
    printf("Maximum : %.2f\n", max);
    printf("Minimum : %.2f\n", min);
    printf("Admis : %d\n", admis);
}
  
```

```
    } while (notes[i] < 0 || notes[i] > 20);
}

/* 3) Parcours du tableau : somme, max, min, admis */
somme = 0;
max = notes[0];      /* initialisation a la premiere note */
min = notes[0];
admis = 0;
for (i = 0; i < n; i++){
    somme = somme + notes[i];
    if (notes[i] > max) max = notes[i];
    if (notes[i] < min) min = notes[i];
    if (notes[i] >= 10) admis = admis + 1;
}

/* 4) Affichage des resultats */
printf("\n--- BULLETIN STATISTIQUE DE LA CLASSE ---\n");
printf("Moyenne de la classe : %.2f / 20\n", somme / n);
printf("Note maximale      : %.2f\n", max);
printf("Note minimale      : %.2f\n", min);
printf("Nombre d'admis      : %d sur %d\n", admis, n);

getch(); /* maintient la fenetre ouverte */
return 0;
}
```

Étape 5 — Compilation et erreurs typiques rencontrées. Lors de la première compilation, les groupes rencontrent en général : « missing terminating " character » (guillemet oublié dans un `printf`), « expected ; before } » (point-virgule manquant), ou un programme qui « plante » parce que le `&` a été omis dans `scanf`. Chaque message indique le numéro de ligne : on lit, on comprend, on corrige, on recompile.

Étape 6 — Validation avec le jeu d'essai. On saisit $n = 5$, puis les notes 12, 0, 20, 7,5, puis 25 : le programme refuse 25 (message reposé) et accepte 14. Affichage obtenu : moyenne = 10,70, maximum = 20,00, minimum = 0,00, admis = 3 sur 5. Les résultats attendus sont confirmés : le programme est validé. Le cas limite $n = 1$ avec la note 10 donne moyenne = 10,00 et 1 admis : correct.

ATTENTION Pièges à éviter dans ce programme

- Oublier le `&` dans `scanf` : comportement imprévisible, souvent un plantage.
- Écrire `somme / n` avec `somme` déclarée en `int` : la division entière tronquerait la moyenne (53,5/5 donnerait 10 au lieu de 10,70). D'où le choix de `float` pour `somme`.
- Initialiser `max` à 0 : si toutes les notes étaient... cela fonctionne ici car les notes sont positives, mais l'initialisation propre et générale est `max = notes[0]`.
- Utiliser `<= n` dans la boucle `for` : la case `notes[n]` n'existe pas, les indices s'arrêtent à $n - 1$.

7.6 Bilan de l'activité

Cette activité a permis de mettre en évidence plusieurs acquis essentiels. D'abord, la **chaîne complète de production** d'un programme : analyse du besoin réel (le travail de Mme Etoundi), algorithme sur papier, traduction en C, compilation, correction guidée par les messages de l'IDE, validation par un jeu d'essai. Ensuite, le **choix raisonné des boucles** : `do...while` s'impose pour toute saisie contrôlée car la question précède le test, tandis que `for` convient au parcours d'un tableau dont la taille est connue. L'activité a aussi consolidé la maîtrise des caractères % (formats d'affichage et de saisie) et & (adressage dans `scanf`), ainsi que le rôle des bibliothèques. Enfin, le bilan collectif a montré qu'un même problème admet plusieurs solutions correctes, et que la qualité d'un programme se juge à sa lisibilité (commentaires, messages clairs), à sa robustesse (refus des valeurs invalides) et à sa validation par des tests incluant les cas limites — autant de réflexes professionnels que tu réutiliseras dans tous tes futurs programmes.

8 MÉTHODES ET EXERCICES RÉSOLUS

9 PROGRAMMATION EN LANGAGE C

9.1 Installer un IDE C et écrire la structure d'un programme

MÉTHODE Écrire et exécuter son premier programme C

Pour passer de l'algorithme au programme C exécutable, suis toujours la même démarche :

1. Installer un IDE adapté (Code ::Blocks avec le compilateur MinGW/GCC, ou Dev-C++). Vérifier à l'ouverture que le compilateur est détecté (menu Settings → Compiler dans Code ::Blocks).
2. Créer un nouveau fichier source : File → New → Source File, puis l'enregistrer immédiatement avec l'extension `.c` (ex. `bonjour.c`).
3. Écrire la structure minimale : inclusion de la bibliothèque `stdio.h`, fonction `main` qui retourne un entier, accolades ouvrante et fermante.
4. Compiler (touche F9 ou Build) : le compilateur traduit le code source en langage machine et signale les erreurs avec leur numéro de ligne.
5. Exécuter (F10 ou Run) et vérifier l'affichage dans la console.

Réflexe à acquérir : tout programme C contient une et une seule fonction `main`; chaque instruction se termine par un point-virgule `;` ; le programme se termine par `return 0;` qui signale au système que tout s'est bien passé.

EXEMPLE Structure minimale d'un programme C

```
#include <stdio.h> /* Bibliothèque standard d'entrées/sorties */

int main(void)      /* Point d'entrée : fonction principale */
{
    /* Corps du programme : instructions terminées par ; */
    printf("Bonjour !\n");
    return 0;        /* Code de retour : 0 = succès */
}
```

EXERCICE RÉSOLU 8 Premier programme : message de bienvenue

Énoncé. Le lycée bilingue de Bafoussam veut afficher à l'ouverture de ses postes informatiques le message suivant sur deux lignes :

Bienvenue au Lycée de Bafoussam
Bon travail !

Écrire le programme C complet correspondant, en précisant la bibliothèque utilisée.

SOLUTION

Solution détaillée.

1. Analyse. Le programme ne fait qu'afficher du texte : il faut donc la fonction de sortie `printf`, qui est déclarée dans la bibliothèque `stdio.h` (standard input/output). On l'inclut avec la directive `#include <stdio.h>`.
2. Passage à la ligne. En C, la séquence d'échappement `\n` provoque un retour à la ligne à l'intérieur d'une chaîne de caractères.
3. Rédaction. La structure minimale impose la fonction `main` et le `return 0;`.

```
#include <stdio.h> /* bibliotheque des entrees/sorties */

int main(void)
{
    printf("Bienvenue au Lycee de Bafoussam\n");
    printf("Bon travail !\n");
    return 0; /* le programme s'est termine normalement */
}
```

Vérification. À la compilation (F9), aucune erreur ; à l'exécution, la console affiche exactement les deux lignes demandées. Chaque instruction se termine bien par ; et les accolades délimitent le corps de `main`. Conclusion. Le programme ci-dessus répond au besoin : il illustre la structure minimale de tout programme C (inclusion de bibliothèque, fonction `main`, instructions terminées par ;, `return 0;`).

LESAVAIS-TU ? Naissance du C et contexte camerounais

Le langage C a été créé au début des années 1970 par Dennis Ritchie aux laboratoires Bell (USA) pour réécrire le système d'exploitation UNIX. Sa portabilité et sa proximité du matériel en ont fait le langage de référence pour les systèmes embarqués, les pilotes (drivers) et les OS. Au Cameroun, la majorité des logiciels embarqués dans les équipements CAMTEL (routeurs, commutateurs), les automates industriels des usines (Alucam, SOSUCAM) et les microcontrôleurs des stations météo de l'ASECNA sont écrits en C. Maîtriser le C, c'est accéder au « cœur » de la machine.

9.2 Inclure les bibliothèques et utiliser les fonctions d'entrée/sortie

PROPRIÉTÉ Bibliothèques standard du programme de Première

Bibliothèque	Fonctions principales	Directive d'inclusion
stdio.h	printf, scanf, getchar, puts	#include <stdio.h>
stdlib.h	system, exit, atoi, rand, malloc	#include <stdlib.h>
math.h	sqrt, pow, cos, sin, floor	#include <math.h>
conio.h	getch, clrscr, kbhit	#include <conio.h>

Note importante : `conio.h` n'est pas standard ISO C (héritage Borland/Turbo C) mais disponible sous Windows dans `Code::Blocks/Dev-C++`. `math.h` nécessite l'option de liaison `-lm` avec GCC (ex. : `gcc prog.c -o prog -lm`).

MÉTHODE Utiliser printf, scanf et getch avec les bonnes bibliothèques

1. Choisir la bibliothèque selon le besoin (voir tableau ci-dessus).
2. Inclure la bibliothèque en tête de fichier : `#include <nom.h>`.
3. Afficher avec `printf("format", variables)` en utilisant les spécificateurs de format :
 - `%d` ou `%i` : entier signé (int)
 - `%f` : réel simple précision (float)
 - `%lf` : réel double précision (double) — obligatoire pour `scanf` avec double
 - `%c` : caractère (char)
 - `%s` : chaîne de caractères (tableau de `char` terminé par `\0`)
 - `%p` : adresse mémoire (pointeur, hexadécimal)
4. Saisir avec `scanf("format", &variable)` : le caractère `&` (esperluette) donne l'adresse de la variable, indispensable pour que `scanf` puisse la modifier. Exception : un tableau de caractères (chaîne) s'écrit sans `&` car le nom du tableau représente déjà son adresse.
5. Faire une pause en fin de programme avec `getch()` ; (bibliothèque `conio.h`) pour laisser la console ouverte avant sa fermeture automatique.

Réflexe : le nombre de `%` dans le format doit être exactement égal au nombre de variables passées, dans le même ordre et avec le bon type.

EXEMPLE Spécificateurs de format avancés pour printf

```
printf("Entier : %d\n", 42);           // 42
printf("Reel : %.2f\n", 3.14159);     // 3.14 (2 decimales)
printf("Reel : %10.2f\n", 3.14);      // largeur 10, aligne a droite
printf("Caractere : %c\n", 'A');      // A
printf("Chaine : %s\n", "Cameroun"); // Cameroun
printf("Adresse : %p\n", &variable); // 0x7ffd...
printf("Pourcent : %%\n");           // % (il faut doubler le %)
```

EXERCICE RÉSOLU 9 Calcul de la moyenne d'un élève

Énoncé. Pour préparer les bulletins du collège de Ngaoundéré, on veut un programme qui demande les notes de mathématiques et de physique d'un élève (réels), calcule leur moyenne et l'affiche avec deux décimales. Écrire ce programme complet.

SOLUTION

Solution détaillée.

1. Analyse. Il y a des saisies (`scanf`), un calcul et un affichage (`printf`) : la bibliothèque `stdio.h` suffit. On ajoute `conio.h` pour la pause finale.
2. Choix des types. Les notes sont des réels : type `float`. La moyenne est aussi un réel.
3. Formule. $\text{moyenne} = \frac{\text{note1} + \text{note2}}{2}$.
4. Rédaction. Pour `scanf`, on écrit `¬e1` car `note1` est une variable simple. Pour l'affichage à deux décimales, on utilise le format `%.2f`.

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    float note1, note2, moyenne;

    printf("Note de mathématiques : ");
    scanf("%f", &note1);      /* & = adresse de la variable */
    printf("Note de physique : ");
    scanf("%f", &note2);

    moyenne = (note1 + note2) / 2.0; /* 2.0 force la division réelle */

    printf("Moyenne = %.2f\n", moyenne);
    getch();                  /* pause : la console reste ouverte */
    return 0;
}
```

Vérification. Avec `note1 = 12` et `note2 = 15` : $\text{moyenne} = \frac{12 + 15}{2} = 13,5$, affiché `13.50`. Le format `%.2f` arrondit bien à deux décimales. Conclusion. Le programme lit deux réels grâce à `scanf` avec l'opérateur d'adresse `&`, calcule la moyenne et l'affiche formatée : il répond exactement au cahier des charges.

EXPÉRIENCE TP guidé : Installation et premier programme

Objectif. Installer Code ::Blocks, créer un projet, compiler et exécuter un programme affichant vos nom, prénom et série. Matériel/Logiciel. Poste Windows, Internet (pour télécharger `codeblocks-20.03mingw-setup.exe` sur sourceforge.net ou le site officiel). Manipulation.

1. Télécharger et installer Code ::Blocks avec MinGW (fichier `mingw` dans le nom). Accepter l'association des fichiers `.c/.cpp`.
2. Lancer Code ::Blocks → Create a new project → Console application → Go → Language C → Nom du projet : `PremierProg` → Dossier : `C:\TP_C` → Finish.
3. Dans `main.c`, remplacer le code par celui de l'exemple `bonjour.c` ci-dessus (avec vos infos).
4. Appuyer sur F9 (Build) : vérifier `0 error(s)`, `0 warning(s)` dans l'onglet Build log.
5. Appuyer sur F10 (Run) : une console noire s'ouvre avec votre message. Appuyer sur une touche pour la fermer (`getch`).

Observation. Si `gcc` n'est pas trouvé : Settings → Compiler → Toolchain executables → Auto-detect. Interprétation. Vous venez de réaliser le cycle complet : édition → compilation → édition de liens → exécution.

9.3 Variables, types et mode d'adressage

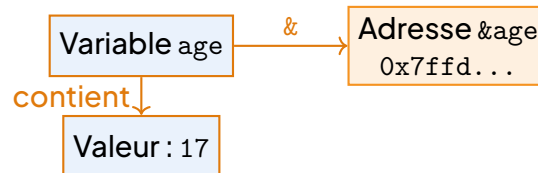
DÉFINITION Variable en C

Une **variable** est une zone de mémoire nommée, de taille fixe selon son **type**, dont le contenu peut varier pendant l'exécution. Elle est caractérisée par :

- un nom (identificateur) : lettres, chiffres, `_`, commence par une lettre, pas d'accent, pas de mot réservé ;
- un type : `int`, `float`, `double`, `char` (et dérivés `short`, `long`, `unsigned`) ;
- une valeur stockée à une adresse mémoire (ex. `0x7ffd3a1c`).

MÉTHODE Déclarer des variables et comprendre l'adressage

1. Choisir le type selon la nature de la donnée : `int` (entier, ex. un âge, un compteur), `float` ou `double` (réel, ex. un prix, une note), `char` (caractère, ex. une mention 'A').
2. Déclarer avant d'utiliser : toute variable doit être déclarée (type + nom) avant sa première utilisation, en début de bloc (norme C90 exigée au Probatoire). Ex. `int age = 17;`.
3. Nommer correctement : lettres, chiffres et `_`, en commençant par une lettre; pas d'accent, pas d'espace, pas de mot réservé (`int`, `for`, `while`...).
4. Comprendre le mode d'adressage : chaque variable occupe une case mémoire identifiée par une adresse. L'opérateur `&` (esperluette) donne l'adresse d'une variable : c'est le passage par adresse qui permet à une fonction comme `scanf` de modifier la variable. Sans `&`, on transmet seulement la valeur (passage par valeur) et la fonction ne peut pas modifier l'originale.
5. Initialiser systématiquement les variables (surtout les compteurs et accumulateurs : `somme = 0;`) pour éviter les valeurs aléatoires (garbage values).



Relation entre variable, valeur et adresse mémoire (opérateur `&`).

EXERCICE RÉSOLU 10 Échange de deux variables et adresses

Énoncé. On considère le programme suivant :

```
#include <stdio.h>

int main(void)
{
    int a = 5, b = 9, temp;
    temp = a;
    a = b;
    b = temp;
    printf("a = %d et b = %d\n", a, b);
    printf("adresse de a : %p\n", (void*)&a);
    return 0;
}
```

1) Donner l'affichage produit par la première ligne `printf`. 2) Expliquer le rôle du caractère

& dans la deuxième ligne. 3) Que se passerait-il si l'on supprimait la variable `temp` en écrivant `a = b; b = a;` ?

SOLUTION

Solution détaillée.

1. Trace d'exécution. On suit les valeurs pas à pas :

Instruction	a	b	temp
Initialisation	5	9	?
<code>temp = a;</code>	5	9	5
<code>a = b;</code>	9	9	5
<code>b = temp;</code>	9	5	5

L'affichage est donc : `a = 9` et `b = 5`. L'échange est réussi grâce à la variable intermédiaire.

2. Rôle de `&`. Le caractère `&` est l'opérateur adresse de : `&a` désigne l'adresse mémoire de la variable `a` (mode d'adressage direct). Le format `%p` permet d'afficher cette adresse (en hexadécimal, par exemple `0x7ffd...`). Le cast `(void*)` est recommandé pour `%p`. C'est ce même opérateur qui permet à `scanf` de savoir où ranger la valeur saisie.
3. Sans variable temporaire. Avec `a = b;` puis `b = a;` : après la première instruction, `a` vaut 9 et la valeur 5 est définitivement perdue ; la deuxième instruction recopie donc 9 dans `b`. Résultat : `a = 9` et `b = 9`. L'échange échoue.

Conclusion. L'affichage est `a = 9` et `b = 5`; `&` donne l'adresse d'une variable ; l'échange de deux variables exige obligatoirement une troisième variable temporaire.

9.4 Les structures répétitives : `for`, `while` et `do...while`

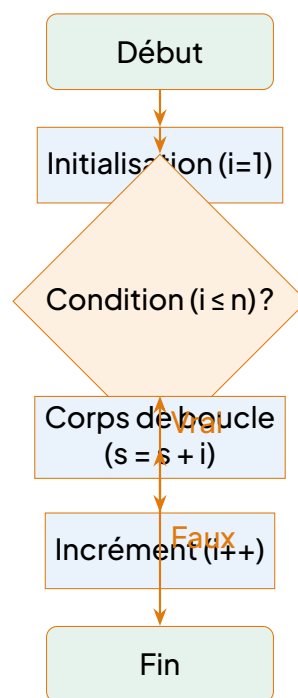
PROPRIÉTÉ Syntaxe des boucles en C

Boucle	Syntaxe
<code>for</code>	<code>for (init ; cond ; incr) { instr ; }</code>
<code>while</code>	<code>while (cond) { instr ; }</code>
<code>do...while</code>	<code>do { instr ; } while (cond) ;</code> (point-virgule final obligatoire)

Dans les trois cas, `instr` peut être une instruction simple ou un bloc `{ ... }`. Les accolades sont obligatoires si plusieurs instructions forment le corps de la boucle.

MÉTHODE Choisir et rédiger une boucle en C

1. Choisir la boucle selon le contexte :
 - nombre d'itérations connu à l'avance $\Rightarrow$ `for`;
 - répétition sous condition, avec éventuellement zéro passage $\Rightarrow$ `while`;
 - corps exécuté au moins une fois (menu, saisie contrôlée) $\Rightarrow$ `do...while`.
2. Respecter les trois éléments de toute boucle : initialisation du compteur, condition d'arrêt, modification du compteur dans le corps (sinon boucle infinie).
3. Initialiser les accumulateurs (`somme = 0` ; `produit = 1`;) avant la boucle, jamais dedans.
4. Vérifier la boucle sur un petit cas (par exemple $n = 3$) avec un tableau de trace.



Organigramme (algorithme) d'une boucle `for` / `while`.

EXERCICE RÉSOLU 11 Somme des n premiers entiers — trois versions

Énoncé. Une coopérative de quartier à Douala veut calculer la somme $S = 1 + 2 + \dots + n$ pour un entier n saisi au clavier. 1) Écrire le programme avec une boucle `for`. 2) Réécrire la boucle en `while`. 3) Donner la valeur affichée pour $n = 4$ en détaillant la trace.

SOLUTION

Solution détaillée.

1. Version `for`. Le nombre d'itérations (n) est connu à l'avance : la boucle `for` est la plus naturelle. L'accumulateur `s` est initialisé à 0 avant la boucle.

```
#include <stdio.h>

int main(void)
{
    int n, i, s = 0;
    printf("Entrer n : ");
    scanf("%d", &n);
    for (i = 1 ; i <= n ; i++){
        s = s + i;
    }
    printf("S = %d\n", s);
    return 0;
}
```

2. Version `while`. On sort l'initialisation et l'incrément de l'en-tête :

```
i = 1;                /* Initialisation */
while (i <= n) {      /* Condition */
    s = s + i;        /* Corps */
    i = i + 1;        /* Incrément : OBLIGATOIRE */
}
```

3. Trace pour $n = 4$.

Tour	i	Condition $i \leq 4$	s
avant boucle	1	—	0
1	1	vraie	$0 + 1 = 1$
2	2	vraie	$1 + 2 = 3$
3	3	vraie	$3 + 3 = 6$
4	4	vraie	$6 + 4 = 10$
sortie	5	fausse	10

Le programme affiche $S = 10$. Contrôle par la formule : $S = \frac{n(n+1)}{2} = \frac{4 \times 5}{2} = 10$. Les deux résultats concordent.

Conclusion. Les boucles `for` et `while` donnent le même résultat ; pour $n = 4$, le programme affiche $S = 10$, conforme à la formule $\frac{n(n+1)}{2}$.

EXERCICE RÉSOLU 12 Saisie contrôlée avec `do...while`

Énoncé. Pour un logiciel de cantine scolaire, on veut obliger l'utilisateur à saisir une note comprise entre 0 et 20 : tant que la valeur est hors de cet intervalle, la question est répétée. Écrire le fragment de programme correspondant et justifier le choix de la boucle.

SOLUTION

Solution détaillée.

1. Choix de la boucle. La question doit être posée au moins une fois (on ne peut pas tester la note avant de l'avoir saisie) : c'est le cas typique du `do...while`, dont le corps s'exécute avant le test. Un `while` simple exigerait une initialisation artificielle de la note.
2. Condition de poursuite. On recommence tant que la saisie est invalide, c'est-à-dire tant que `note < 0` ou `note > 20` (opérateur logique `||`).

```
#include <stdio.h>

int main(void)
{
    float note;
    do {
        printf("Entrer une note entre 0 et 20 : ");
        scanf("%f", &note);
    } while (note < 0 || note > 20); /* ; obligatoire ici */

    printf("Note validee : %.2f\n", note);
    return 0;
}
```

Vérification. Si l'utilisateur tape 25 puis -3 puis 14,5 : les deux premières valeurs sont rejetées (condition vraie), la troisième est acceptée (condition fausse) et le programme affiche `Note validee : 14.50`. Conclusion. Le `do...while` garantit au moins une saisie et ne laisse passer que les notes de l'intervalle `[0 ; 20]` : c'est la structure adaptée à toute saisie contrôlée.

EXPÉRIENCE TP guidé : Boucles et trace papier

Objectif. Maîtriser la trace d'exécution (tableau de valeurs) pour valider une boucle. Consigne. Écrire un programme qui calcule $P = 1 \times 2 \times \dots \times n$ (factorielle) pour n saisi ($n \leq 12$ pour tenir dans un `int`). Tester avec $n = 5$. Étapes.

1. Écrire l'algorithme sur papier (variables : `n`, `i`, `p=1`, boucle `for` de 1 à `n`).
2. Coder dans l'IDE.
3. Avant d'exécuter, remplir le tableau de trace pour $n = 5$:

i	p (avant)	p (après $p=p*i$)
1	1	1
2	1	2
3	2	6
4	6	24
5	24	120

4. Compiler, exécuter, comparer l’affichage (120) à la trace.

Compétence. La trace papier est l’outil indispensable pour détecter les erreurs de logique (boucle infinie, borne erronée) avant même de compiler.

9.5 Déclarer et exploiter un tableau à une dimension

DÉFINITION Tableau à une dimension

Un **tableau** est une collection ordonnée de variables de même type, stockées en mémoire de manière contiguë, accessibles par un **indice** (entier) commençant à 0.

- **Déclaration** : `type nom[taille];` (ex. `float notes[30];`).
- **Accès** : `nom[indice]` (ex. `notes[0]` première note, `notes[29]` dernière).
- Taille fixe à la compilation (allocation statique).

MÉTHODE Manipuler un tableau en C

1. Déclarer le tableau avec son type, son nom et sa taille entre crochets : `float notes[30];` (30 cases pour les notes d’une classe).
2. Se souvenir que les indices commencent à 0 : un tableau de taille n a des cases numérotées de 0 à $n-1$. La case `notes[n]` n’existe pas : y accéder est une erreur classique (buffer overflow).
3. Remplir le tableau avec une boucle `for` de 0 à $n-1$: `scanf("%f", &tab[i]);` (le `&` reste nécessaire car `tab[i]` est une variable simple).
4. Traiter (somme, moyenne, maximum, minimum) avec une deuxième boucle, en initialisant les accumulateurs avant la boucle ; pour un maximum/minimum, initialiser avec `tab[0]`.
5. Initialiser à la déclaration si les valeurs sont connues : `int t[5] = {4, 7, 1, 9, 3};`

EXERCICE RÉSOLU 13 Notes d'une classe : moyenne et maximum

Énoncé. Le professeur d'informatique du lycée de Maroua saisit les notes de 5 élèves dans un tableau. Écrire un programme C qui : remplit le tableau, calcule et affiche la moyenne de la classe, puis affiche la note maximale. Donner le résultat pour les notes {12; 8; 15; 10; 14}.

SOLUTION

Solution détaillée.

1. Analyse. Tableau de 5 réels : `float notes[5];`, indices de 0 à 4. Trois phases : remplissage, somme puis moyenne, recherche du maximum.
2. Initialisations. `somme = 0` avant la boucle de calcul; `max = notes[0]` avant la recherche (initialiser le maximum à 0 serait faux si toutes les notes étaient négatives, et dangereux en général).

```
#include <stdio.h>

int main(void)
{
    float notes[5], somme = 0, moyenne, max;
    int i;

    for (i = 0 ; i < 5 ; i++){           /* remplissage */
        printf("Note de l'eleve %d : ", i + 1);
        scanf("%f", &notes[i]);
    }

    for (i = 0 ; i < 5 ; i++){           /* somme */
        somme = somme + notes[i];
    }
    moyenne = somme / 5.0;

    max = notes[0];                      /* recherche du maximum */
    for (i = 1 ; i < 5 ; i++){
        if (notes[i] > max){
            max = notes[i];
        }
    }

    printf("Moyenne = %.2f\n", moyenne);
    printf("Note maximale = %.2f\n", max);
    return 0;
}
```

Vérification numérique. Avec {12; 8; 15; 10; 14} :

$$\text{somme} = 12 + 8 + 15 + 10 + 14 = 59, \quad \text{moyenne} = \frac{59}{5} = 11,8.$$

Recherche du maximum : `max = 12`, puis `8 < 12`, puis `15 > 12` donc `max = 15`, puis 10 et 14 ne changent rien. Le programme affiche donc Moyenne = 11.80 et Note maximale = 15.00. Conclusion. Le programme remplit le tableau avec des indices de 0 à 4, calcule

une moyenne de 11,80 et trouve un maximum de 15 : les résultats manuels et automatiques concordent.

EXEMPLE Initialisation statique et parcours

```
#include <stdio.h>

int main(void)
{
    int t[5] = {4, 7, 1, 9, 3}; /* Initialisation à la déclaration */
    int i, somme = 0;

    for (i = 0; i < 5; i++) {
        printf("t[%d] = %d\n", i, t[i]);
        somme += t[i];          /* Equivalent a somme = somme + t[i] */
    }
    printf("Somme = %d\n", somme);
    return 0;
}
```

9.6 Compiler, tester et corriger un programme dans l'IDE

MÉTHODE Tester et déboguer un programme C

1. Compiler (F9 sous Code ::Blocks) et lire la première erreur de la liste : les suivantes en découlent souvent. Le compilateur indique le fichier, la ligne et le type d'erreur (error bloquante, warning à corriger aussi).
2. Corriger les erreurs fréquentes : point-virgule manquant, variable non déclarée, parenthèse ou accolade non fermée, = à la place de == dans un test, bibliothèque oubliée (implicit declaration of function).
3. Tester avec un jeu d'essai : valeurs ordinaires, valeurs limites (0, 20 pour une note), cas particuliers ($n = 1$, tableau d'un seul élément).
4. Tracer à la main ou ajouter des `printf` intermédiaires (debug prints) pour localiser une erreur de logique (résultat faux alors que la compilation réussit).
5. Recompiler après chaque correction et conserver une version qui fonctionne avant toute modification.

EXERCICE RÉSOLU 14 Chasse aux erreurs

Énoncé. Un élève a écrit le programme suivant, censé afficher si un entier est pair ou impair. Le compilateur signale plusieurs erreurs. Identifier chaque erreur, la corriger et donner l’affichage obtenu pour la saisie $n = 7$.

```
#include <stdio.h>

int main()
{
    int n
    printf("Entrer un entier : ");
    scanf("%d", n);
    if (n % 2 = 0)
        printf("pair\n");
    else
        printf("impair\n");
    return 0;
}
```

SOLUTION

Solution détaillée.

1. Erreur 1 — ligne 4. Il manque le point-virgule après la déclaration : `int n`; (message typique : `expected ';' before 'printf'`).
2. Erreur 2 — ligne 6. Il manque l’opérateur d’adresse dans `scanf` : il faut `scanf("%d", &n)`; . Sans `&`, `scanf` reçoit la valeur de `n` au lieu de son adresse : comportement imprévisible (souvent un plantage segmentation fault).
3. Erreur 3 — ligne 7. `=` est l’affectation, pas la comparaison : il faut `if (n % 2 == 0)`. Avec `=`, l’expression `n % 2 = 0` est d’ailleurs invalide car on ne peut pas affecter à une expression (lvalue required).

Programme corrigé :

```
#include <stdio.h>

int main(void)
{
    int n;
    printf("Entrer un entier : ");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("pair\n");
    else
        printf("impair\n");
    return 0;
}
```

Test pour $n = 7$. L’opérateur `%` donne le reste de la division euclidienne : $7 = 2 \times 3 + 1$, donc $7 \% 2 = 1 \neq 0$. La condition est fausse, le programme affiche `impair`, ce qui est correct. Conclusion. Trois erreurs corrigées (point-virgule, `&` dans `scanf`, `==` dans le test); pour $n = 7$

le programme affiche `impair`, conforme au calcul $7 \bmod 2 = 1$.

ATTENTION Les 5 erreurs qui coûtent des points au Probatoire

1. Oublier le `&` dans `scanf`. Sans esperluette, la saisie ne modifie pas la variable et le programme plante ou donne des résultats absurdes. Seule exception : les chaînes de caractères (tableaux), qui s'écrivent sans `&` (ex. `scanf("%s", nom);`).
2. Confondre `=` et `==`. `if (a = 5)` affecte 5 à `a` et la condition est toujours vraie (valeur 5 non nulle) : le test est faussé silencieusement. La comparaison s'écrit toujours `==`.
3. Indices de tableau hors limites. Un tableau de taille `n` s'indice de 0 à `n-1`. Écrire `for (i = 1 ; i <= n ; i++)` sur un tableau de taille `n` provoque un débordement (buffer overflow) : la boucle correcte est `for (i = 0 ; i < n ; i++)`.
4. Oublier d'initialiser un accumulateur ou le réinitialiser dans la boucle. `somme = 0;` se place avant la boucle ; placé dedans, la somme ne contient que la dernière valeur. De même, oublier l'incrément dans un `while` crée une boucle infinie.
5. Négliger la syntaxe de fin d'instruction et des bibliothèques. Point-virgule manquant (y compris le ; après le `while` d'un `do...while`), bibliothèque non incluse (`math.h` pour `sqrt`, `conio.h` pour `getch`) ou division entière non maîtrisée (`5 / 2` vaut 2, pas 2,5 : écrire `5.0 / 2` ou utiliser des variables `float`).

EXERCICE 6 Exercices d'entraînement — Probatoire

1. QCM. Quel est l'affichage du code suivant ?

```
int i, s = 0;
for (i = 0; i < 3; i++) s += i;
printf("%d", s);
```

A. 3 B. 6 C. 2 D. 0

2. Code à trous. Compléter pour afficher la table de multiplication de 7 (de 7×1 à 7×10).

```
int i, n = 7;
for (i = 1; i <= 10; i++) {
    printf("%d x %d = %d\n", n, i, _____);
}
```

3. Algorithme → C. Traduire en C : « Saisir 10 entiers dans un tableau `T`. Afficher le nombre de valeurs positives. »
4. Erreur logique. Le programme ci-dessous doit calculer la moyenne de 3 notes. Pour-

quoi affiche-t-il toujours 0.00?

```
float n1, n2, n3, moy;
scanf("%f%f%f", &n1, &n2, &n3);
moy = (n1 + n2 + n3) / 3;
printf("%.2f", moy);
```

CORRIGÉ Exercices d'entraînement

1. B. Trace : $i = 0 \rightarrow s = 0$; $i = 1 \rightarrow s = 1$; $i = 2 \rightarrow s = 3$; $i = 3$ stop. $s = 3$. Attention : la condition est $i < 3$, donc i prend 0, 1, 2. Somme = $0 + 1 + 2 = 3$. (Réponse A).
2. Trou : $n * i$ (ou $7 * i$).
3. Traduction :

```
#include <stdio.h>
int main(void){
    int T[10], i, cp = 0;
    for (i = 0; i < 10; i++){
        printf("T[%d] = ", i); scanf("%d", &T[i]);
    }
    for (i = 0; i < 10; i++){
        if (T[i] > 0) cp++;
    }
    printf("Positifs : %d\n", cp); return 0;
}
```

4. Erreur : Division entière! 3 est un `int`, la somme de `float` est promue en `float`, mais la division par un `int` donne un `float`... Attendez. En C, `float / int → float`. Le code fonctionne pour la division. Mais si les notes étaient des `int`, la division serait entière. Ici, le piège classique est l'oubli de `&` dans `scanf` (mais ils y sont) ou l'initialisation. Correction du piège : Le code donné est correct si les notes sont `float`. Si l'énoncé disait `int n1...`, alors `/3` ferait une division entière. Pour être sûr : écrire `3.0` ou `3.0f`.

À RETENIR Bilan de la leçon N10 — Programmation en C

- IDE : Code ::Blocks + MinGW (GCC). Cycle : Éditer (F9) → Compiler → Exécuter (F10).
- Structure : `#include <stdio.h> / int main(void) { ... return 0; }`.
- Bibliothèques : `stdio.h` (E/S), `stdlib.h` (utilitaires), `math.h` (maths, linker `-lm`), `conio.h` (`getch` `pause`).
- E/S : `printf("format", vars), scanf("format", &vars)`. Spécificateurs : `%d`, `%f`, `%lf`, `%c`, `%s`, `%p`. `&` = adresse (sauf tableaux).

- **Variables** : Déclaration en début de bloc, typage fort (`int`, `float`, `double`, `char`), initialisation obligatoire.
- **Boucles** : `for` (nb connu), `while` (0 ou +), `do...while` (1 au moins). Initialiser avant, incrémenter dans, tracer pour valider.
- **Tableaux** : Indices 0 à $n-1$. Remplissage + Traitement = deux boucles distinctes. Max/-Min init avec `tab[0]`.
- **Débogage** : Lire la 1ère erreur, corriger les 5 pièges classiques (`&`, `==`, bornes, `init`, `;`).

10 EXERCICES

10.1 Je vérifie mes connaissances

EXERCICE 7 QCM : à chaque question, une seule bonne réponse

Recopie la lettre correspondant à la bonne réponse.

- Un IDE est :
 - un langage de programmation ;
 - un environnement de développement intégré regroupant éditeur, compilateur et débogueur ;
 - un système d'exploitation.
- Pour utiliser les fonctions `printf` et `scanf`, on écrit en tête du programme :
 - `#include <math.h>;`
 - `#include <conio.h>;`
 - `#include <stdio.h>.`
- Dans l'instruction `scanf("%d", &n);`, le caractère `&` placé devant `n` sert à :
 - indiquer l'adresse mémoire de la variable `n` ;
 - doubler la valeur de `n` ;
 - déclarer la variable `n`.
- La déclaration correcte d'un tableau de 10 entiers nommé `notes` est :
 - `int notes(10);;`
 - `int notes[10];;`
 - `tableau notes[10];.`

EXERCICE 8 Vrai ou faux ? Justifie ta réponse

Pour chacune des affirmations suivantes, réponds par vrai ou faux en justifiant brièvement.

- La boucle `do...while` exécute son corps au moins une fois, même si la condition est fausse dès le départ.
- En C, le premier élément d'un tableau `t` déclaré par `int t[5];` est `t[1]`.

3. La fonction `sqrt` (racine carrée) est disponible après inclusion de la bibliothèque `math.h`.
4. Le format `%f` dans un `printf` permet d'afficher un entier de type `int`.

EXERCICE 9 Définitions

Définis brièvement chacun des termes suivants, en donnant un exemple d'utilisation en langage C :

1. bibliothèque;
2. paramètre (d'une fonction comme `printf` ou `scanf`);
3. mode d'adressage (passage par valeur et passage par adresse);
4. IDE.

EXERCICE 10 Trace d'exécution

On considère l'extrait de programme C suivant :

```
int t[5] = {4, 7, 2, 9, 5};
int i, s = 0;
for (i = 0; i < 5; i = i + 2) {
    s = s + t[i];
}
printf("%d", s);
```

1. Construis le tableau d'évolution des variables `i` et `s` à chaque passage dans la boucle.
2. Quelle valeur est affichée à l'écran à la fin du programme ?

10.2 Je m'entraîne

EXERCICE 11 Somme des n premiers entiers

Écris un programme C complet qui :

1. saisit un entier `n` strictement positif (avec contrôle de saisie : tant que `n` est négatif ou nul, la saisie est recommencée);
2. calcule et affiche la somme $1 + 2 + \dots + n$ en utilisant une boucle `for`;
3. calcule et affiche la même somme en utilisant une boucle `while`.

EXERCICE 12 Âges des élèves

Le professeur d'informatique du lycée de Bafoussam veut connaître l'âge extrême de ses élèves. Écris un programme C qui :

1. déclare un tableau de 8 entiers ;
2. saisit les âges de 8 élèves dans ce tableau à l'aide d'une boucle ;
3. détermine puis affiche l'âge du plus jeune élève et celui du plus âgé.

EXERCICE 13 Menu géométrique

Écris un programme C qui affiche le menu suivant :

1. Surface du carré
 2. Surface du cercle
 0. Quitter

Le programme doit :

1. répéter l'affichage du menu à l'aide d'une boucle `do...while` jusqu'à ce que l'utilisateur choisisse 0 ;
2. pour le choix 1, saisir le côté c et afficher la surface c^2 (utiliser `pow` de `math.h`) ;
3. pour le choix 2, saisir le rayon r et afficher la surface πr^2 (prendre $\pi = 3,14159$).

EXERCICE 14 Chasse aux erreurs

Le programme suivant, censé saisir un prix hors taxe et afficher le prix TTC (TVA de 19,25 %), contient exactement cinq erreurs. Identifie-les et réécris le programme corrigé.

```
#include <stdio.h>

int main() {
    float prixHT, prixTTC;
    printf("Prix HT : ");
    scanf("%d", prixHT)
    prixTTC = prixHT * 1.1925;
    printf("Prix TTC = %d FCFA\n", prixTTC);
    return 0;
```

Indication : pense aux points-virgules, au caractère `&`, aux formats d'affichage et aux accolades.

10.3 Je me prépare au Probatoire

EXERCICE 15 Type Probatoire — Moyenne des notes d'une classe

Pour l'examen blanc du lycée de Ngaoundéré, le professeur souhaite calculer la moyenne des notes de ses 10 élèves en informatique. Voici le programme C qu'un élève a commencé à écrire :

```
..... <stdio.h>          // (1)

int main() {
    float notes[...];        // (2)
    float somme = 0, moyenne;
    int i;
    for (i = 0; i < 10; i++) {
        printf("Note de l'eleve %d : ", i + 1);
        scanf("...", ...(notes[i])); // (3) et (4)
        somme = somme + notes[i];
    }
    moyenne = somme / 10;
    printf("Moyenne = ...\n", moyenne); // (5)
    return 0;
}
```

1. Recopie et complète ce programme en remplaçant les pointillés (1) à (5) par ce qui convient : directive d'inclusion, taille du tableau, format de saisie, symbole d'adressage, format d'affichage.
2. Explique le rôle du caractère & utilisé à la ligne de saisie. Que risque-t-il de se passer si on l'oublie ?
3. Le professeur veut maintenant connaître la note la plus élevée de la classe. Modifie le programme complété pour qu'il affiche aussi la meilleure note.
4. Propose une modification pour que chaque note saisie soit obligatoirement comprise entre 0 et 20 (contrôle de saisie avec une boucle `do...while`).

EXERCICE 16 Type Probatoire — Caisse d'une boutique

Mme Etoundi tient une boutique d'articles scolaires à Yaoundé. Pour chaque client, elle saisit les prix des articles achetés, un par un, et elle tape la valeur 0 pour signaler la fin de la vente. On veut écrire un programme C qui l'aide à établir ses factures.

1. Écris un programme C qui, à l'aide d'une boucle `while`, saisit les prix des articles (des nombres réels, en FCFA) jusqu'à ce que la valeur saisie soit 0, puis affiche le montant total de la vente et le nombre d'articles achetés.
2. Mme Etoundi accorde une remise de 5 % si le montant total dépasse 50000 FCFA. Modifie le programme pour afficher le montant de la remise éventuelle et le montant

net à payer.

3. On souhaite maintenant mémoriser les prix dans un tableau de 50 cases maximum. Réécris le programme en stockant chaque prix saisi dans le tableau, puis en calculant le total par une boucle `for` qui parcourt le tableau.
4. Donne la structure générale d'un programme C (bibliothèques, fonction `main`, valeur de retour) en l'illustrant sur ton programme de la question 3.

EXERCICE 17 Type Probatoire — Statistiques au CDI

Le documentaliste du lycée de Maroua a relevé, pendant 7 jours, le nombre d'élèves venus au CDI. Il veut un programme C qui saisit ces 7 nombres dans un tableau, puis affiche : le total des visiteurs de la semaine, la moyenne journalière et le nombre de jours où la fréquentation a dépassé 30 élèves.

1. Définis les termes suivants : IDE, bibliothèque, paramètre.
2. Écris le programme C demandé, en soignant la structure (inclusion des bibliothèques nécessaires, fonction `main`, déclarations).
3. On exécute le programme avec les valeurs suivantes : 25, 32, 18, 41, 30, 35, 27. Construis le tableau de suivi des variables (total, compteur de jours au-dessus de 30) et donne les trois valeurs affichées par le programme.
4. Le documentaliste veut refuser toute saisie négative. Ajoute au programme un contrôle de saisie utilisant une boucle `do...while`, et justifie le choix de cette boucle plutôt que `while`.

11 CORRIGÉS DES EXERCICES

11.1 Corrigés — Je vérifie mes connaissances

CORRIGÉ Exercice 1 — QCM : à chaque question, une seule bonne réponse

1. Réponse b. Un IDE (Integrated Development Environment, environnement de développement intégré) est un logiciel qui regroupe dans une même interface un **éditeur** de code, un **compilateur** et un **débogueur** (exemples : Code ::Blocks, Dev-C++, CodeLite). Ce n'est ni un langage ni un système d'exploitation.
2. Réponse c. Les fonctions `printf` et `scanf` sont déclarées dans la bibliothèque `stdio.h` (standard input/output). On écrit donc `#include <stdio.h>`.
3. Réponse a. Le caractère `&` est l'opérateur d'**adressage** : `&n` désigne l'adresse mémoire de la variable `n`. `scanf` a besoin de cette adresse pour savoir où ranger la valeur saisie (passage par adresse).
4. Réponse b. En C, un tableau se déclare avec des crochets : `int notes[10];`. Les parenthèses servent aux fonctions, et le mot `tableau` n'existe pas en C.

CORRIGÉ Exercice 2 — Vrai ou faux ?

1. Vrai. Dans une boucle `do...while`, le test de la condition est effectué après l'exécution du corps : le corps est donc exécuté au moins une fois, même si la condition est fausse dès le départ. C'est ce qui la distingue de la boucle `while`.
2. Faux. En C, les indices d'un tableau commencent à 0. Pour `int t[5];`, le premier élément est `t[0]` et le dernier est `t[4]`. Accéder à `t[5]` est une erreur (dépassement de tableau).
3. Vrai. La fonction `sqrt` (ainsi que `pow`, `fabs`, etc.) est déclarée dans la bibliothèque `math.h`. Sans la directive `#include <math.h>`, le compilateur signale une erreur.
4. Faux. Le format `%f` est réservé aux nombres réels (`float` ou `double`). Pour un entier de type `int`, on utilise `%d`. Utiliser `%f` avec un `int` produit un affichage erroné.

CORRIGÉ Exercice 3 — Définitions

1. Bibliothèque : ensemble de fonctions prédéfinies, regroupées dans un fichier d'en-tête (`.h`), que l'on rend disponibles dans un programme grâce à la directive `#include`.

Exemple : `#include <math.h>` permet d'utiliser `sqrt(x)` et `pow(a, b)`.

2. **Paramètre :** information transmise à une fonction lors de son appel, placée entre parenthèses, qui précise ce sur quoi la fonction travaille. Exemple : dans `printf("%d", age);`, les paramètres sont la chaîne de format `"%d"` et la variable `age`; dans `scanf("%f", &x);`, les paramètres sont `"%f"` et `&x`.
3. **Mode d'adressage :** manière dont une valeur est transmise à une fonction. Dans le passage par valeur, la fonction reçoit une copie de la valeur (ex. `printf("%d", n);` : `printf` reçoit la valeur de `n`). Dans le passage par adresse, la fonction reçoit l'adresse mémoire de la variable et peut donc la modifier (ex. `scanf("%d", &n);` : `scanf` écrit directement dans `n`).
4. **IDE :** environnement de développement intégré, logiciel regroupant un éditeur de code source, un compilateur et un débogueur, qui permet d'écrire, de compiler, d'exécuter et de tester un programme. Exemple : Code ::Blocks, dans lequel on écrit un fichier `main.c` puis on lance la compilation avec F9.

CORRIGÉ Exercice 4 — Trace d'exécution

1. La boucle fait varier `i` de 2 en 2 : `i` prend les valeurs 0, 2, 4. On ajoute donc `t[0] = 4`, `t[2] = 2` et `t[4] = 5`.

Passage	<code>i</code>	<code>t[i]</code>	<code>s</code> après l'addition
Initialisation	0	—	0
1 ^{er} tour	0	4	$0 + 4 = 4$
2 ^e tour	2	2	$4 + 2 = 6$
3 ^e tour	4	5	$6 + 5 = 11$
Sortie de boucle	6	—	11 (condition <code>i < 5</code> fausse)

2. La valeur affichée à l'écran est 11. (Vérification : $4 + 2 + 5 = 11$.)

11.2 Corrigés — Je m'entraîne

CORRIGÉ Exercice 5 — Somme des n premiers entiers

```
#include <stdio.h>

int main() {
    int n, i, sommeFor = 0, sommeWhile = 0;
```

```

/* Saisie controlee : on recommence tant que n <= 0 */
do{
    printf("Saisir un entier strictement positif : ");
    scanf("%d", &n);
} while (n <= 0);

/* a) Calcul avec la boucle for */
for (i = 1; i <= n; i++){
    sommeFor = sommeFor + i;
}
printf("Avec for : 1 + ... + %d = %d\n", n, sommeFor);

/* b) Calcul avec la boucle while */
i = 1; /* reinitialisation du compteur */
while (i <= n) {
    sommeWhile = sommeWhile + i;
    i++;
}
printf("Avec while : 1 + ... + %d = %d\n", n, sommeWhile);

return 0;
}

```

Remarques. Le contrôle de saisie utilise naturellement un `do...while` car la saisie doit avoir lieu au moins une fois. Il ne faut pas oublier de remettre `i` à 1 avant la boucle `while`, sinon le second calcul part de `i = n + 1` et donne 0. Vérification : pour $n = 4$, on obtient $1 + 2 + 3 + 4 = 10$, ce qui correspond à la formule $\frac{n(n+1)}{2} = \frac{4 \times 5}{2} = 10$.

CORRIGÉ Exercice 6 — Âges des élèves

```

#include <stdio.h>

int main() {
    int ages[8]; /* declaration du tableau de 8 entiers */
    int i, min, max;

    /* Saisie des 8 ages */
    for (i = 0; i < 8; i++){
        printf("Age de l'eleve %d : ", i + 1);
        scanf("%d", &ages[i]);
    }

    /* Initialisation : le premier element est a la fois min et max */
    min = ages[0];
    max = ages[0];

    /* Parcours du reste du tableau */
    for (i = 1; i < 8; i++){
        if (ages[i] < min) {
            min = ages[i];
        }
    }
}

```

```
    }
    if (ages[i] > max) {
        max = ages[i];
    }
}

printf("Age du plus jeune eleve : %d ans\n", min);
printf("Age du plus age eleve : %d ans\n", max);

return 0;
}
```

Point de méthode. On initialise `min` et `max` avec `ages[0]` (et non avec 0, qui risquerait d'être plus petit que tous les âges réels et de fausser le maximum... ou l'inverse), puis on parcourt le tableau à partir de l'indice 1 en comparant chaque élément aux deux bornes.

CORRIGÉ Exercice 7 — Menu géométrique

```
#include <stdio.h>
#include <math.h> /* pour la fonction pow */

int main() {
    int choix;
    float c, r, surface;

    do {
        /* Affichage du menu */
        printf("\n1. Surface du carre\n");
        printf("2. Surface du cercle\n");
        printf("0. Quitter\n");
        printf("Votre choix : ");
        scanf("%d", &choix);

        if (choix == 1) {
            printf("Cote du carre : ");
            scanf("%f", &c);
            surface = pow(c, 2); /* c au carre */
            printf("Surface du carre = %.2f\n", surface);
        }
        else if (choix == 2) {
            printf("Rayon du cercle : ");
            scanf("%f", &r);
            surface = 3.14159 * pow(r, 2); /* pi * r^2 */
            printf("Surface du cercle = %.2f\n", surface);
        }
        else if (choix != 0) {
            printf("Choix invalide, recommencez.\n");
        }
    } while (choix != 0); /* on recommence tant que ce n'est pas 0 */

    printf("Fin du programme.\n");
    return 0;
}
```

```
| }
```

Vérifications. Pour $c = 5$: surface = 25. Pour $r = 2$: surface = $3,14159 \times 4 \approx 12,57$. La boucle `do...while` est bien adaptée : le menu doit s'afficher au moins une fois.

CORRIGÉ Exercice 8 — Chasse aux erreurs

Les cinq erreurs sont :

1. `scanf("%d", prixHT)` : mauvais format, `prixHT` est un `float`, il faut `%f` ;
2. `scanf(..., prixHT)` : il manque le `&` devant `prixHT` (passage par adresse obligatoire pour `scanf`) ;
3. il manque le point-virgule à la fin de la ligne `scanf(...)` ;
4. `printf("... %d ...", prixTTC)` : mauvais format d'affichage, il faut `%f` pour un `float` ;
5. il manque l'accolade fermante `}` de la fonction `main`.

Programme corrigé :

```
#include <stdio.h>

int main() {
    float prixHT, prixTTC;
    printf("Prix HT : ");
    scanf("%f", &prixHT);           /* erreurs 1, 2 et 3 corrigees */
    prixTTC = prixHT * 1.1925;
    printf("Prix TTC = %.2f FCFA\n", prixTTC); /* erreur 4 corrigee : %.2f pour l'affichage monetaire */
    return 0;
}                                   /* erreur 5 corrigee */
```

Vérification. Pour un prix HT de 10000 FCFA : prix TTC = $10000 \times 1,1925 = 11925$ FCFA.

11.3 Corrigés — Je me prépare au Probatoire

CORRIGÉ Exercice 9 — Type Probatoire : moyenne des notes d'une classe

1. Programme complété :

```
#include <stdio.h>           // (1) directive d'inclusion

int main() {
    float notes[10];         // (2) tableau de 10 reels
    float somme = 0, moyenne;
    int i;
    for (i = 0; i < 10; i++) {
```

```

        printf("Note de l'eleve %d : ", i + 1);
        scanf("%f", &notes[i]); // (3) format %f (4) symbole &
        somme = somme + notes[i];
    }
    moyenne = somme / 10;
    printf("Moyenne = %.2f\n", moyenne); // (5) format %f (%.2f pour 2 decimales)
    return 0;
}

```

2. Le caractère `&` donne à `scanf` l'adresse mémoire de la case `notes[i]` : c'est un passage par adresse, qui permet à `scanf` d'écrire la valeur saisie directement dans le tableau. Si on l'oublie, `scanf` reçoit la valeur contenue dans la case (un nombre quelconque) et l'interprète comme une adresse : le comportement est imprévisible, en général le programme plante (erreur d'exécution) ou écrit n'importe où en mémoire.
3. On ajoute une variable `max` initialisée à `notes[0]` après la saisie, puis un parcours comparatif :

```

#include <stdio.h>

int main() {
    float notes[10];
    float somme = 0, moyenne, max;
    int i;
    for (i = 0; i < 10; i++) {
        printf("Note de l'eleve %d : ", i + 1);
        scanf("%f", &notes[i]);
        somme = somme + notes[i];
    }
    moyenne = somme / 10;

    max = notes[0];           /* initialisation au premier element */
    for (i = 1; i < 10; i++) {
        if (notes[i] > max) {
            max = notes[i];
        }
    }

    printf("Moyenne = %.2f\n", moyenne);
    printf("Meilleure note = %.2f\n", max);
    return 0;
}

```

4. Contrôle de saisie avec `do...while` (la saisie est faite au moins une fois, puis recommencée tant qu'elle est invalide) :

```

    for (i = 0; i < 10; i++) {
        do {
            printf("Note de l'eleve %d (0 a 20) : ", i + 1);
            scanf("%f", &notes[i]);
        } while (notes[i] < 0 || notes[i] > 20);
        somme = somme + notes[i];
    }
}

```

Barème indicatif (sur 10) : directives et structure du programme 2 pts ; formats %f et & correctement justifiés 2 pts ; calcul de la moyenne 2 pts ; recherche du maximum (initialisation + parcours) 2 pts ; contrôle de saisie 2 pts.

Points de vigilance : citer le rôle du passage par adresse pour `scanf` ; initialiser `max` avec `notes[0]` et non avec 0 ; justifier le choix du `do...while` (au moins une saisie obligatoire).

CORRIGÉ Exercice 10 — Type Probatoire : caisse d'une boutique

1. Saisie des prix avec une boucle `while` (valeur sentinelle 0) :

```
#include <stdio.h>

int main() {
    float prix, total = 0;
    int nbArticles = 0;

    printf("Prix de l'article (0 pour terminer) : ");
    scanf("%f", &prix);

    while (prix != 0) {
        total = total + prix;
        nbArticles = nbArticles + 1;
        printf("Prix de l'article (0 pour terminer) : ");
        scanf("%f", &prix);
    }

    printf("Nombre d'articles : %d\n", nbArticles);
    printf("Montant total : %.0f FCFA\n", total);
    return 0;
}
```

2. Ajout de la remise de 5 % au-delà de 50000 FCFA, juste avant l'affichage final :

```
float remise = 0, net;
if (total > 50000) {
    remise = total * 0.05;
}
net = total - remise;
printf("Remise : %.0f FCFA\n", remise);
printf("Net à payer : %.0f FCFA\n", net);
```

Vérification : pour un total de 60000 FCFA, $\text{remise} = 60000 \times 0,05 = 3000$ FCFA, $\text{net} = 57000$ FCFA.

3. Version avec mémorisation dans un tableau de 50 cases (utilisation de `nb` comme compteur d'articles valides) :

```
#include <stdio.h>

int main() {
    float prix[50]; /* tableau de 50 cases maximum */
}
```

```

float total = 0;
int nb = 0, i;
float val;

/* Saisie avec stockage (au plus 50 articles) */
printf("Prix de l'article (0 pour terminer) : ");
scanf("%f", &val);
while (val != 0 && nb < 50) {
    prix[nb] = val; /* on stocke la valeur lue */
    nb = nb + 1; /* nb devient le nombre d'articles saisis */
    printf("Prix de l'article (0 pour terminer) : ");
    scanf("%f", &val);
}

/* Calcul du total par parcours du tableau avec for */
for (i = 0; i < nb; i++) {
    total = total + prix[i];
}

printf("Nombre d'articles : %d\n", nb);
printf("Montant total : %.0f FCFA\n", total);
return 0;
}

```

Attention : la condition `nb < 50` empêche le dépassement des 50 cases du tableau (indices 0 à 49). La variable `nb` compte exactement le nombre d'articles valides, ce qui simplifie la boucle de sommation `for (i=0; i<nb; i++)`.

4. Structure générale d'un programme C, illustrée sur le programme de la question 3 :

- Inclusion des bibliothèques : `#include <stdio.h>` rend disponibles `printf` et `scanf`;
- fonction principale : `int main() { ... }`, point d'entrée du programme, qui contient les déclarations de variables (`float prix[50];`, etc.) puis les instructions;
- valeur de retour : `return 0;` signale au système que le programme s'est terminé correctement ; le type de retour de `main` est `int`.

Barème indicatif (sur 10) : boucle `while` avec valeur sentinelle 0 correcte 2,5 pts ; compteur et total 1,5 pts ; remise conditionnelle 2 pts ; tableau + boucle `for` de cumul 2 pts ; structure générale expliquée 2 pts.

Points de vigilance : la première saisie doit précéder le `while` (amorçage) ; ne pas compter le 0 final comme un article ; justifier `%f` et `&` ; mentionner le risque de dépassement de tableau.

CORRIGÉ Exercice 11 — Type Probatoire : statistiques au CDI

1. IDE : environnement de développement intégré, logiciel regroupant éditeur, compilateur et débogueur (ex. Code ::Blocks). Bibliothèque : fichier d'en-tête regroupant

des fonctions prédéfinies, inclus par `#include` (ex. `stdio.h` pour `printf`). Paramètre : information transmise à une fonction lors de son appel (ex. dans `scanf("%d", &x)` ; les paramètres sont `"%d"` et `&x`).

2. Programme demandé :

```
#include <stdio.h>

int main() {
    int visiteurs[7];    /* frequentation des 7 jours */
    int i, total = 0, nbJours30 = 0;
    float moyenne;

    for (i = 0; i < 7; i++) {
        printf("Visiteurs du jour %d : ", i + 1);
        scanf("%d", &visiteurs[i]);
        total = total + visiteurs[i];
        if (visiteurs[i] > 30) {
            nbJours30 = nbJours30 + 1;
        }
    }

    moyenne = (float) total / 7; /* conversion pour garder les decimales */

    printf("Total de la semaine : %d visiteurs\n", total);
    printf("Moyenne journaliere : %.2f\n", moyenne);
    printf("Jours a plus de 30 eleves : %d\n", nbJours30);
    return 0;
}
```

3. Tableau de suivi avec les valeurs 25, 32, 18, 41, 30, 35, 27 :

Jour i	visiteurs[i]	total	> 30?	nbJours30
0	25	25	non	0
1	32	57	oui	1
2	18	75	non	1
3	41	116	oui	2
4	30	146	non (30 n'est pas > 30)	2
5	35	181	oui	3
6	27	208	non	3

Valeurs affichées : total = 208 visiteurs; moyenne = $\frac{208}{7} \approx 29,71$; nombre de jours à plus de 30 élèves = 3. Attention au piège : la valeur 30 ne compte pas, car la condition est strictement supérieure à 30.

4. Contrôle de saisie :

```
for (i = 0; i < 7; i++) {
    do {
```

```
        printf("Visiteurs du jour %d : ", i + 1);  
        scanf("%d", &visiteurs[i]);  
    } while (visiteurs[i] < 0);  
    total = total + visiteurs[i];  
    if (visiteurs[i] > 30){  
        nbJours30 = nbJours30 + 1;  
    }  
}
```

Justification du choix : la boucle `do...while` exécute son corps au moins une fois, ce qui correspond exactement au besoin : chaque jour doit être saisi une première fois, puis la saisie n'est recommencée que si elle est invalide. Avec un `while`, il faudrait soit faire une première saisie avant la boucle (duplication de code), soit initialiser artificiellement la variable à une valeur négative.

Barème indicatif (sur 10) : définitions 1,5 pt ; structure du programme (inclusions, `main`, déclarations) 1,5 pt ; saisie et cumuls 2 pts ; tableau de suivi exact 2 pts ; trois résultats affichés 1 pt ; contrôle de saisie et justification du `do...while` 2 pts.

Points de vigilance : condition stricte `> 30` ; conversion (`float`) pour obtenir une moyenne décimale (sinon $208/7 = 29$ en division entière) ; justification explicite du choix de la boucle.

12 FICHE BILAN



Carte mentale : Leçon N10 – Programmation en langage C (1ère C/D/E)

FICHE BILAN L'essentiel à connaître pour l'examen

12.1 Définitions clés à connaître par cœur

- **IDE (Environnement de Développement Intégré)** : Logiciel tout-en-un (éditeur,

compilateur, débogueur). Ex. : Code ::Blocks (recommandé au Cameroun), Dev-C++, VS Code + GCC.

- **Bibliothèque** : Fichier d'en-tête (.h) contenant des prototypes de fonctions prédéfinies. Inclusion par `#include <nom.h>`.
- **Programme C minimal** : Une fonction `main()` retournant un `int`.
- **Mode d'adressage (&)** : Opérateur « adresse de ». Indispensable pour `scanf` afin de modifier la variable appelante.
- **Spécificateur de format (%)** : Marqueur dans la chaîne de contrôle de `printf/scanf` indiquant le type (%d, %f, %c, %s, %lf).
- **Tableau à une dimension** : Collection ordonnée d'éléments de même type, indices de 0 à N-1. Accès : `T[i]`.

12.2 Syntaxes essentielles (mémoriser le squelette)

Concept	Syntaxe / Règle	Exemple court
Structure de base	<pre>#include <stdio.h> int main() { // instructions return 0; }</pre>	<pre>#include <stdio.h>int main() { printf("Bonjour"); return 0;}</pre>
Affichage (printf)	<pre>printf("format", var1, var2); %d (entier), %f (réel), %c (caractère), %s (chaine), \n (re- tour ligne)</pre>	<pre>printf("Age : %d\n", age);</pre>
Saisie (scanf)	<pre>scanf("format", &var); Obligatoire : & devant la va- riable (sauf tableau/chaine).</pre>	<pre>scanf("%f", &note);</pre>
Boucle for	<pre>for (init; cond; incr) { bloc; } Idéale quand le nombre de tours est connu.</pre>	<pre>for(int i=0; i<10; i++) printf("%d ", i);</pre>
Boucle while	<pre>while (cond) { bloc; } Test avant en- trée. Peut ne jamais s'exécu- ter.</pre>	<pre>while(n>0) { n--; }</pre>
Boucle do...while	<pre>do { bloc; } while (cond); Test après en- trée. S'exécute au moins une fois.</pre>	<pre>do { scanf("%d",&x); } while(x<0);</pre>
Déclaration tableau	<pre>type nom[taille]; Taille constante (ou variable en C99/VLA). Indices : 0 à taille-1.</pre>	<pre>int notes[30];float prix[5] = {100, 200};</pre>

12.3 Méthodes express

MÉTHODE Écrire et tester un programme C complet

1. Ouvrir l'IDE (Code::Blocks) → File > New > Empty File → Sauver .c.
2. Écrire les includes nécessaires (`stdio.h` minimum).
3. Écrire `int main() { ... return 0;}`.
4. Déclarer les variables en haut du bloc (ancien standard C89) ou au plus près (C99).
5. Coder l'algorithme : `printf` pour guider, `scanf` (%... &var) pour lire, boucles pour répéter, tableaux pour stocker.
6. Compiler : F9 (Build) ou Ctrl+F9 (Build & Run). Lire les errors (rouge) et warnings (jaune) dans le log bas.
7. Corriger : erreur de syntaxe (point-virgule, accolade), erreur de format (%d vs %f), & oublié.
8. Tester : valeurs normales, valeurs limites (0, négatif, grand nombre), entrée invalide (lettre au lieu de nombre).

MÉTHODE Parcourir un tableau pour calcul (somme, max, recherche)

1. Initialiser l'accumulateur (`sum=0, max=T[0], trouve=0`).
2. Boucle `for(int i=0; i<N; i++)`.
3. À l'intérieur : mise à jour (`sum += T[i];` ou `if(T[i]>max) max=T[i];`).
4. Après la boucle : résultat dans la variable.

À RETENIR Checklist avant le Bac – Leçon N10 : Programmation C

- ☐ Je sais installer et lancer Code::Blocks (ou Dev-C++) et créer un fichier .c.
- ☐ J'écris la structure de base `#include <stdio.h> / int main() { ... return 0;}` sans regarder.
- ☐ Je distingue `printf` (affichage, pas de &) et `scanf` (saisie, & obligatoire sur variables simples).

- ☐ Je choisis le bon spécificateur : `%d` (int), `%f` (float), `%lf` (double), `%c` (char), `%s` (chaîne).
- ☐ J'écris correctement les trois boucles : `for`, `while`, `do...while` (attention au ; final du `do-while`).
- ☐ Je déclare un tableau : `type Tab[Taille]` ; et j'accède aux éléments par `Tab[i]` avec `i` de 0 à `Taille-1`.
- ☐ Je sais parcourir un tableau avec une boucle `for` pour faire une somme, une recherche, un tri simple ou un affichage.
- ☐ J'inclus `math.h` pour `sqrt`, `pow` et `conio.h` pour `getch()` si besoin de pause console.
- ☐ Je compile avec F9, je lis les erreurs (ligne, type : `syntax error`, `format mismatch`, `undeclared variable`) et je corrige.
- ☐ Je teste mon programme avec des cas limites : boucle qui ne tourne pas (`while(0)`), saisie négative, tableau vide, division par zéro évitée.
- ☐ Je connais le rôle du préprocesseur (`#include`, `#define`), du compilateur (`C → .o`), de l'éditeur de liens (`.o + lib → .exe`).
- ☐ Je gère les entrées invalides : vérifier le retour de `scanf` (nombre d'éléments lus) pour éviter les boucles infinies.

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+